

대분류/19
전기·전자

중분류/03
전자기기개발

소분류/08
로봇개발

세분류/01
로봇하드웨어설계

능력단위/20

NCS학습모듈

로봇 주제어장치(MCU) 하드웨어 회로설계

LM1903080120_22v3



교육부

[NCS학습모듈 활용 시 유의 사항]

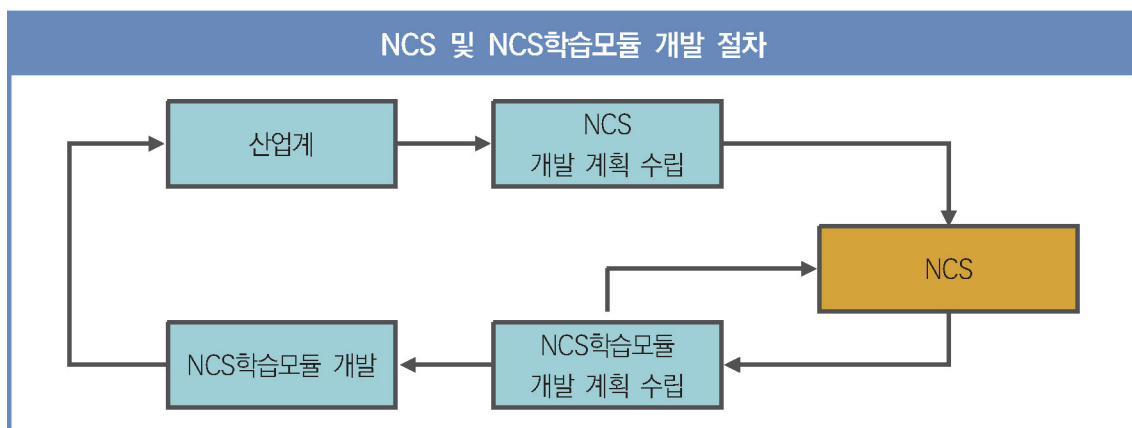
1. NCS학습모듈은 교육훈련기관에서 출처를 명시하고 교육적 목적으로 활용할 수 있습니다. 다만, NCS학습모듈에는 국가(교육부)가 저작권재산권 일체를 보유하지 않은 저작물(출처가 표기된 도표·사진·삽화·도면 등)이 포함되어 있으므로, 이러한 저작물의 변형·각색·복제·공연·배포 및 공중 송신 등과 이러한 저작물을 활용한 2차적 저작물을 작성하려면 반드시 원작자의 동의를 받아야 합니다.
2. NCS학습모듈은 개발 당시의 산업 및 교육 현장을 반영하여 집필하였으므로, 현재 적용되는 법령·지침·표준 및 교과 내용 등과 차이가 있을 수 있습니다. NCS학습모듈 활용 시 법령·지침·표준 및 교과 내용의 개정 사항과 통계의 최신성 등을 확인하시기를 바랍니다.
3. NCS학습모듈은 산업 현장에서 요구되는 능력을 교육훈련기관에서 학습할 수 있게 구성한 자료입니다. 다만, NCS학습모듈 지면의 한계상 대표적 예시(예: 활용도 또는 범용성이 높은 제품, 서비스) 중심으로 집필하였음을 이해하시기를 바랍니다.

NCS학습모듈의 이해

※ 본 NCS학습모듈은 「NCS 국가직무능력표준」사이트(<http://www.ncs.go.kr>) 에서 확인 및 다운로드할 수 있습니다.

I NCS학습모듈이란?

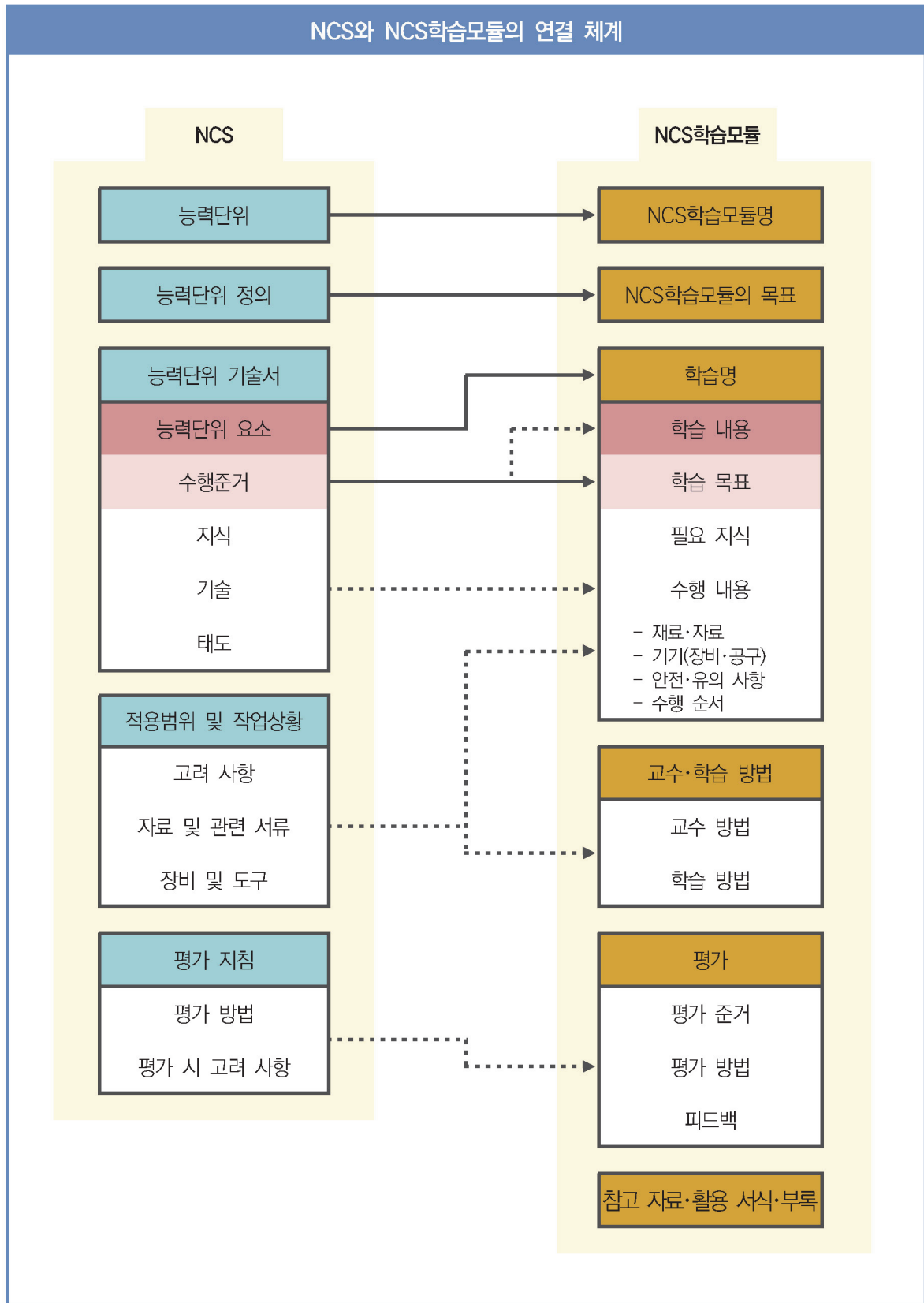
- 국가직무능력표준(NCS: National Competency Standards)이란 산업현장에서 직무를 수행하기 위해 요구되는 지식·기술·소양 등의 내용을 국가가 산업부문별·수준별로 체계화한 것으로 산업현장의 직무를 성공적으로 수행하기 위해 필요한 능력(지식, 기술, 태도)을 국가적 차원에서 표준화한 것을 의미합니다.
- 국가직무능력표준(이하 NCS)이 현장의 ‘직무 요구서’라고 한다면, **NCS학습모듈은 NCS의 능력단위를 교육훈련에서 학습할 수 있도록 구성한 ‘교수·학습 자료’입니다.** NCS학습모듈은 구체적 직무를 학습할 수 있도록 이론 및 실습과 관련된 내용을 상세하게 제시하고 있습니다.



○ NCS학습모듈은 다음과 같은 특징을 가지고 있습니다.

- 첫째, NCS학습모듈은 산업계에서 요구하는 직무능력을 교육훈련 현장에 활용할 수 있도록 성취목표와 학습의 방향을 명확히 제시하는 가이드라인의 역할을 합니다.
- 둘째, NCS학습모듈은 특성화고, 마이스터고, 전문대학, 4년제 대학교의 교육기관 및 훈련기관, 직장교육기관 등에서 표준교재로 활용할 수 있으며 교육과정 개편 시에도 유용하게 참고할 수 있습니다.

○ NCS와 NCS학습모듈 간의 연결 체계를 살펴보면 아래 그림과 같습니다.



II NCS학습모듈의 체계

○ NCS학습모듈은 1. NCS학습모듈의 위치, 2. NCS학습모듈의 개요, 3. NCS학습모듈의 내용 체계, 4. 참고 자료, 5. 활용서식/부록으로 구성되어 있습니다.

1. NCS학습모듈의 위치

○ NCS학습모듈의 위치는 NCS 분류 체계에서 해당 학습모듈이 어디에 위치하는지를 한 눈에 볼 수 있도록 그림으로 제시한 것입니다.

[NCS-학습모듈의 위치]		
대분류	문화·예술·디자인·방송	
중분류	문화콘텐츠	
소분류	문화콘텐츠제작	
세분류		
방송콘텐츠제작	능력단위	학습모듈명
영화콘텐츠제작	프로그램 기획	프로그램 기획
음악콘텐츠제작	아이템 선정	아이템 선정
광고콘텐츠제작	자료 조사	자료 조사
게임콘텐츠제작	프로그램 구성	프로그램 구성
애니메이션 콘텐츠제작	캐스팅	캐스팅
만화콘텐츠제작	제작계획	제작계획
캐릭터제작	방송 미술 준비	방송 미술 준비
스마트문화앱 콘텐츠제작	방송 리허설	방송 리허설
영사	야외촬영	야외촬영
	스튜디오 제작	스튜디오 제작
	...	...

학습모듈은

NCS 능력단위 1개당 1개의 학습모듈 개발을 원칙으로 합니다. 그러나 필요에 따라 고용단위 및 교과단위를 고려하여 능력단위 몇 개를 묶어 1개 학습모듈로 개발할 수 있으며, NCS 능력단위 1개를 여러 개의 학습모듈로 나누어 개발할 수도 있습니다.

2. NCS학습모듈의 개요

○ NCS학습모듈의 개요는 학습모듈이 포함하고 있는 내용을 개략적으로 설명한 것으로

학습모듈의 목표, 선수학습, 학습모듈의 내용 체계, 핵심 용어 로 구성되어 있습니다.

학습모듈의 목표	해당 NCS 능력단위의 정의를 토대로 학습 목표를 작성한 것입니다.
선수학습	해당 학습모듈에 대한 효과적인 교수·학습을 위하여 사전에 이수해야 하는 학습모듈, 학습 내용, 관련 교과목 등을 기술한 것입니다.
학습모듈의 내용 체계	해당 NCS 능력단위요소가 학습모듈에서 구조화된 체계를 제시한 것입니다.
핵심 용어	해당 학습모듈의 학습 내용, 수행 내용, 설비·기자재 등 가운데 핵심적인 용어를 제시한 것입니다.

제작계획 학습모듈의 개요

학습모듈의 목표

본격적인 촬영을 준비하는 단계로서, 촬영 대본을 확정하고 제작 스태프를 조직하며 촬영 장비와 촬영 소품을 준비할 수 있다.

선수학습

제작 준비(LM0803020105_13v1), 섭외 및 제작스태프 구성(LM0803020104_13v1), 촬영 제작(LM0803020106_13v1), 촬영 장비 준비(LM0803040204_13v1.4), 미술 디자인 협의하기(LM0803040203_13v1.4)

학습모듈의 내용체계

학습	학습 내용	NCS 능력단위 요소	
		코드번호	요소 명칭
1. 촬영 대본 확정하기	1-1. 촬영 구성안 검토와 수정	0803020114_16.3.1	촬영 대본 확정하기
2. 제작 스태프 조직하기	2-1. 기술 스태프 조직 2-2. 미술 스태프 조직 2-3. 전문 스태프 조직	0803020114_16.3.2	제작 스태프 조직하기
3. 촬영 장비 계획하기	3-1. 촬영 장비 점검과 준비	0803020114_16.3.3	촬영 장비 계획하기
4. 촬영 소품 계획하기	4-1. 촬영 소품 목록 작성 4-2. 촬영 소품 제작 의뢰	0803020114_16.3.4	촬영 소품 계획하기

핵심 용어

촬영 구성안, 제작 스태프, 촬영 장비, 촬영 소품

학습모듈의 목표는

학습자가 해당 학습모듈을 통해 성취해야 할 목표를 제시한 것으로, 교수자는 학습자가 학습모듈의 전체적인 내용흐름을 파악하도록 지도할 수 있습니다.

선수학습은

교수자 또는 학습자가 해당 학습모듈을 교수·학습하기 이전에 이수해야 하는 교과목 또는 학습모듈(NCS 능력단위) 등을 표기한 것입니다. 따라서 교수자는 학습자가 개별 학습, 자기 주도 학습, 방과 후 활동 등 다양한 방법을 통해 이수할 수 있도록 지도하는 것을 권장합니다.

핵심 용어는

해당 학습모듈을 대표하는 주요 용어입니다. 학습자가 해당 학습모듈을 통해 학습하고 평가받게 될 주요 내용을 알 수 있습니다. 「NCS 국가직무능력표준」 사이트(www.ncs.go.kr)의 색인(찾아보기) 중 하나로 이용할 수 있습니다.

3. NCS학습모듈의 내용 체계

○ NCS학습모듈의 내용은 크게 **학습**, **학습 내용**, **교수·학습 방법**, **평가**로 구성되어 있습니다.

학습	해당 NCS 능력단위요소 명칭을 사용하여 제시한 것입니다. 학습은 크게 학습 내용, 교수·학습 방법, 평가로 구성되며 해당 NCS 능력단위의 능력단위 요소별 지식, 기술, 태도 등을 토대로 내용을 제시한 것입니다.
학습 내용	학습 내용은 학습 목표, 필요 지식, 수행 내용으로 구성되며, 수행 내용은 재료·자료, 기기(장비·공구), 안전·유의 사항, 수행 순서, 수행 tip으로 구성된 것입니다. 학습모듈의 학습 내용은 실제 산업현장에서 이루어지는 업무활동을 표준화된 프로세스에 기반하여 다양한 방식으로 반영한 것입니다.
교수·학습 방법	학습 목표를 성취하기 위한 교수자와 학습자 간, 학습자와 학습자 간 상호 작용이 활발하게 일어날 수 있도록 교수자의 활동 및 교수 전략, 학습자의 활동을 제시한 것입니다.
평가	평가는 해당 학습모듈의 학습 정도를 확인할 수 있는 평가 준거 및 평가 방법, 평가 결과의 피드백 방법을 제시한 것입니다.

학습 1	촬영 대본 확정하기
학습 2	제작 스태프 조직하기
학습 3	촬영 장비 계획하기
학습 4	촬영 소품 계획하기

2-1. 기술 스태프 조직

학습 목표

- 프로그램 제작에 적합한 기술 스태프를 조직할 수 있다.

필요 지식 /

1. 기술 스태프의 구성
 프로그램의 장르에 따라 구성하는 기술 스태프는 많은 차이가 있다. 같은 장르의 프로그램이라도 그 형식이나 내용, 규모에 따라서 구성되는 기술 스태프의 종류와 인원 수는 천차만별이다.

1. 스튜디오 프로그램
 토크쇼, 종합 구성, 예능과 같은 스튜디오 프로그램은 부조정실과 스튜디오를 사용하여 제작하기 때문에 많은 기술 스태프가 필요하다.

학습은
 해당 NCS 능력단위요소 명칭을 사용하여 제시하였습니다. 하나의 학습은 일반교과의 '대단원'에 해당되며, 학습모듈을 구성하는 가장 큰 단위가 됩니다. 또한 하나의 직무를 수행하기 위한 가장 기본적인 단위로 사용할 수 있습니다.

학습 내용은
 NCS 능력단위요소별 수행준거를 기준으로 제시하였습니다. 일반교과의 '중단원'에 해당합니다.

학습 목표는
 학습 내용을 이수할 때 학습자가 갖춰야 할 행동 수준을 의미합니다. 따라서 수업시간의 과목 목표로 활용할 수 있습니다.

필요 지식은
 해당 NCS의 지식을 토대로 학습에 대한 이해와 성과를 제고하기 위해 반드시 알아야 할 주요 지식을 제시하였습니다. 필요 지식은 수행에 꼭 필요한 핵심 내용을 위주로 제시하여 교수자의 역할이 매우 중요하며, 이후 수행 순서와 연계하여 교수·학습으로 진행할 수 있습니다.

수행 내용 / 기술 스태프 구성표 작성하기

재료·자료

- 방송프로그램 제작 기획서 및 방송 대본, 콘티(continuity), 제작 일정, 운용표
- 장비 및 시설, 제작 시설 배정 의뢰서 및 배정표, 방송 기술 스태프 데이터베이스(DB) 자료

기기(장비·공구)

- 컴퓨터 등

안전·유의 사항

- 프로그램의 내용과 제작 방법을 분석하고, 각 스태프들의 역할을 신중하게 검토한다.

수행 순서

- ① 방송 대본이나 콘티(continuity), 큐 시트를 분석하고, 프로그램의 내용적 특성, 제작 과정에 대한 자료를 수집한다.
- ② 프로그램 제작 방법을 결정한다.
 1. 스튜디오 녹화를 할 것인가, 야외 촬영을 할 것인가 검토한다.

수행 tip

- 스태프의 결정은 스태프 간의 호흡을 중요시하여 선정해야 프로그램의 질을 향상시킬 수 있다.

수행 내용은

해당 학습모델에서 제시한 내용 중 기술(skill)을 습득하기 위한 실습과제로 활용할 수 있습니다.

재료·자료는

수행 내용을 수행하는데 필요한 재료 및 준비물로 실습 시 활용할 수 있습니다.

기기(장비·공구)는

수행 내용에 필요한 기본적인 장비 및 도구를 제시하였습니다. 제시된 기기 외에도 수행에 필요한 다양한 도구나 장비를 활용할 수 있습니다.

안전·유의사항은

수행 내용을 수행하는 데 있어 안전상 주의해야 할 점 및 유의사항을 제시하였습니다. 실습 시 유념해야 하며, NCS의 고려사항도 추가적으로 활용할 수 있습니다.

수행 순서는

실습 과제의 진행 순서로 활용할 수 있습니다.

수행 tip은

수행 내용에서 실습을 용이하게 할 수 있는 아이디어를 제시하였습니다. 수행 tip은 지도상의 안전 및 유의사항 외에 전반적으로 적용되는 주안점 및 수행 과제 목적에 대한 보충설명, 추가사항 등으로 활용할 수 있습니다.

학습2 교수·학습 방법

교수 방법

- 방송 프로그램의 기술적 요소, 미술 구성 요소, 특수 촬영에 대해 설명한다.
- 방송 프로그램 제작에서 각 기술 스태프의 역할에 대해 설명한다.
- 방송 프로그램을 분석하고 필요한 기술 스태프를 구성할 수 있도록 지시한다.

학습 방법

- 방송 프로그램의 기술적 요소, 미술 구성 요소, 특수 촬영에 대해서 알아본다.
- 프로그램 제작에 필요한 기술 스태프의 역할을 이해하고, 기술 스태프 구성표를 작성한다.

교수·학습 방법은

학습 목표를 성취하는 데 필요한 교수 방법과 학습 방법을 제시하였습니다.

교수 방법은

해당 학습 활동에 필요한 학습 내용, 학습 내용과 관련된 자료명, 자료 형태, 수행 내용의 진행 방식 등에 대하여 제시하였습니다. 또한 학습자의 수업참여도 제고 방법 및 수업 진행상 유의사항 등도 제시하였습니다. 선수학습이 필요한 학습을 학습자가 숙지하였는지 교수자가 확인하는 과정으로 활용할 수도 있습니다.

학습 방법은

해당 학습 활동에 필요한 학습자의 자기주도 학습 방법을 제시하였습니다. 또한 학습자가 숙달해야 할 실기 능력과 학습 과정에서 주의해야 할 사항 등도 제시하였습니다. 학습자가 학습을 이수하기 전 반드시 숙지해야 할 기본 지식을 학습하였는지 스스로 확인하는 과정에 활용할 수 있습니다.

학습2

평 가

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준 상 중 하
기술 스태프 조직	- 프로그램 제작에 적합한 기술 스태프를 조직할 수 있다.	
미술 스태프 조직	- 프로그램 제작에 적합한 미술 스태프를 조직할 수 있다.	
전문 스태프 조직	- 프로그램 특수 촬영을 위한 전문 스태프를 조직할 수 있다.	

평가 방법

- 사례 연구

학습 내용	평가 항목	성취수준 상 중 하
기술 스태프 조직	- 프로그램에서 기술적 요소의 파악 여부 - 기술 스태프의 역할 파악 여부 - 프로그램에 필요한 기술 스태프 구성표 작성 능력	

피드백

- 사례 연구
 - 프로그램을 선택하여 기술 스태프, 미술 스태프, 전문 스태프 구성표를 예시와 같이 작성하였는지 개인별 능력을 평가한 후, 그 결과를 모든 학습자에게 공유하도록 한다.

평가는

NCS 능력단위의 평가 방법과 평가 시 고려사항을 준용하여 작성합니다. 교수자와 학습자가 평가 항목별 성취수준 확인 시 활용할 수 있습니다.

평가 준거는

학습자가 학습을 어느 정도 성취하였는지 평가하기 위한 기준을 제시하고 있습니다. 학습 목표와 연계하여 단위수업 시간에 평가 항목 별 성취수준을 평가하는 데 활용할 수 있습니다.

평가 방법은

NCS 능력단위의 평가 방법을 참고하였으며, 평가 준거에 따른 평가 방법을 2개 이상 제시합니다. 평가 방법의 종류는 포트폴리오, 문제해결 시나리오, 서술형 시험, 논술형 시험, 사례 연구, 평가자 체크리스트, 작업장 평가 등이 있으며, NCS 능력단위 요소 별 수행 수준을 평가하는 데 가장 적절한 방법을 선정하여 활용할 수 있습니다.

피드백은

평가 후에 학습자들에게 평가 결과를 피드백하여 학습 목표를 달성하는 데 활용할 수 있습니다.

4. 참고 자료

참 고 자 료

- 교육부(2013). 섭외 및 제작스태프 구성(LM0803020104_13v1). 한국직업능력개발원.

참고 자료는

해당 학습모듈에 제시된 인용 자료의 출처를 제시하였습니다. 교수·학습의 과정에서 참고로 활용할 수 있습니다.

5. 활용 서식/부록

활 용 서 식

스튜디오 기술 스태프 구성표

직종	이름	연락처	소속	특이사항	비고
기술감독					
조명감독					

활용 서식은

평가 서식, 실습 시트 등 교수·학습 시 활용할 수 있는 다양한 서식들로 구성하였습니다. 수행에서 평가에 이르기까지 필요한 서식을 해당 모듈의 특성에 맞춰 개발하거나 기존의 양식을 활용하여 제시하였습니다.

부 록

[디지털 텔레비전 방송프로그램 음량 등에 관한 기준]

제정 2014. 11. 29. 미래창조과학부 고시 제2014-87호

제1장 총칙

제1조(목적) 이 고시는 방송법 제70조의2제1항에 따라 방송사업자가 디지털 텔레비전 방송프로그램 및 방송광고의 음량을 일정하게 유지하기 위해 필요한 사항을 규정함을 목적으로 한다.

부록은

활용 서식 이외에 교수·학습 과정에서 참고할 수 있는 자료가 있는 경우 제시하였습니다.

[NCS-학습모듈의 위치]

대분류	전기·전자
중분류	전자기기개발
소분류	로봇개발

세분류		
로봇하드웨어설계	능력단위	학습모듈명
로봇기구개발	로봇 입출력 인터페이스 하드웨어 설계	로봇 입출력 인터페이스 하드웨어 설계
로봇소프트웨어개발	로봇 전원부 설계	로봇 전원부 설계
로봇지능개발	로봇 전장 설계	로봇 전장 설계
로봇유지보수	로봇 하드웨어 제작	로봇 하드웨어 제작
로봇안전인증	로봇 하드웨어 시험평가	로봇 하드웨어 시험평가
	로봇 하드웨어 유지보수	로봇 하드웨어 유지보수
	로봇 시스템 사양 설계	로봇 시스템 사양 설계
	로봇 하드웨어 아키텍처 설계	로봇 하드웨어 아키텍처 설계
	로봇 하드웨어 아키텍처 구조 설계	로봇 하드웨어 아키텍처 구조 설계
	로봇 액추에이터 드라이버 개념 설계	로봇 액추에이터 드라이버 개념 설계
	로봇 액추에이터 드라이버 회로 설계	로봇 액추에이터 드라이버 회로 설계
	로봇 모션 제어기 하드웨어 구조 설계	로봇 모션 제어기 하드웨어 구조 설계
	로봇 모션 제어기 회로 설계	로봇 모션 제어기 회로 설계

로봇 주제어 장치(MCU) 하드웨어 개념 설계	로봇 주제어 장치(MCU) 하드웨어 개념 설계
로봇 주제어 장치(MCU) 하드웨어 회로 설계	로봇 주제어 장치(MCU) 하드웨어 회로 설계
로봇 센서 신호처리 개념 설계	로봇 센서 신호처리 개념 설계
로봇 센서 신호처리부 설계	로봇 센서 신호처리부 설계

차 례

학습모듈의 개요	1
----------	---

학습 1. 로봇 주제어 장치(MCU) 하드웨어 회로 설계하기

1-1. 로봇 주제어 장치(MCU) 하드웨어 회로 설계	3
• 교수 · 학습 방법	40
• 평가	41

학습 2. 로봇 주제어 장치(MCU) 하드웨어 검증하기

2-1. 로봇 주제어 장치(MCU) 하드웨어 검증	43
• 교수 · 학습 방법	70
• 평가	71

참고 자료	73
-------	----

로봇 주제어 장치(MCU) 하드웨어 회로 설계 학습모듈의 개요

학습모듈의 목표

개념설계와 주제어 장치(MCU) 규격을 바탕으로 주제어 장치(MCU) 하드웨어의 회로를 설계하고 검증할 수 있다.

선수학습

전자 공학 개론, 통신 회로 실험, 디지털 논리회로의 기초와 응용, 로봇 제어부 설계, 로봇 하드웨어 개발 실무

학습모듈의 내용체계

학습	학습 내용	NCS 능력단위 요소	
		코드번호	요소 명칭
1. 로봇 주제어 장치(MCU) 하드웨어 회로 설계하기	1-1. 로봇 주제어 장치(MCU) 하드웨어 회로 설계	1903080120_22v3.1	로봇 주제어 장치(MCU) 하드웨어 회로 설계하기
2. 로봇 주제어 장치(MCU) 하드웨어 검증하기	2-1. 로봇 주제어 장치(MCU) 하드웨어 검증	1903080120_22v3.2	로봇 주제어 장치(MCU) 하드웨어 검증하기

핵심 용어

서빙 로봇, 물류로봇, 자율 주행, AI, MCU, 요구 사항 명세, 통신, 검증

1-1. 로봇 주제어 장치(MCU) 하드웨어 회로 설계

학습 목표

- 선정된 MCU 처리부와 주변기기의 사양을 확인하고 부품을 배치할 수 있다.
- 결정된 메모리용량, 신호처리, 데이터 타이밍 인터페이스를 확인할 수 있다.
- 부품의 배치에 따라 신호의 간섭과 노이즈를 고려하여 MCU부 회로를 설계할 수 있다.
- MCU 회로설계에 관련된 설계도면을 관리할 수 있다.

필요 지식 /

① 회로 설계의 일반적인 진행 단계

아래에서 설명하는 단계는 일반적인 회로 설계의 프로세스를 나타내며, 실제 프로젝트에 따라 상세한 절차는 달라질 수 있다. 설계를 손쉽게 진행할 수 있게 도와주는 회로 설계 도구와 시뮬레이션 도구를 사용하면 설계 과정을 시각화하고 효율적으로 진행할 수 있다.

1. 요구 사항 분석

- (1) 회로의 목적과 기능을 이해하는 것을 포함하여 설계를 위한 요구 사항을 이해하고 문서화한다.
- (2) 통신 프로토콜, 데이터 속도, 신호 형식, 전력 요구 사항 등과 같은 사항을 고려한다.

2. 블록 다이어그램

- (1) 시스템을 논리적인 블록으로 나누고 각 블록 간의 상호 작용을 표시하는 블록 다이어그램을 작성한다.
- (2) 각 블록 간의 인터페이스를 정의한다.
- (3) 각 블록의 기능, 데이터 흐름 및 제어 신호를 나타낸다.

3. 회로 설계

- (1) 회로를 설계하기 위해 전체 시스템에 관한 전체적인 아키텍처를 계획하고 회로 도표를 작성한다.
- (2) 통신 프로토콜과 신호 처리를 위한 회로 요소를 선택하고 구성한다. 필요한 센서, 앰프, 필

터, ADC(아날로그-디지털 변환기), DAC(디지털-아날로그 변환기) 등을 포함한다.

(3) 회로 요소 간의 연결과 신호의 크기, 주파수, 전원 공급을 고려하여 회로를 구성한다.

4. 스키마 설계

(1) 회로 설계를 위한 회로 도표 또는 스키마를 작성한다.

(2) 회로 도표에서는 다이오드, 저항, 커패시터 등 회로 요소의 연결과 값 등을 나타낸다.

5. PCB 레이아웃

(1) 스키마를 기반으로 PCB 레이아웃을 디자인하여 회로 구성 요소를 PCB 상에 배치하고, 연결 선과 패턴을 디자인한다.

(2) 회로 요소의 위치, 신호 및 전원 트레이스 경로, 신호 노이즈와 그라운드 등을 고려하여 PCB 디자인을 진행한다.

6. 시뮬레이션 및 검증

(1) 설계된 회로와 PCB 레이아웃을 시뮬레이션 도구를 사용하여 검증한다.

(2) 전압, 전류, 주파수 응답 등을 확인하고 회로의 안정성을 평가한다.

(3) 시뮬레이션을 통해 신호 노이즈, 신호 왜곡, 신호 간 간섭 등을 확인하고 문제를 해결한다.

7. 제작 및 테스트

(1) 완성된 PCB 디자인에 회로 보호, 재질 선택, 적절한 층 구성 등을 고려하여 샘플을 제조한다.

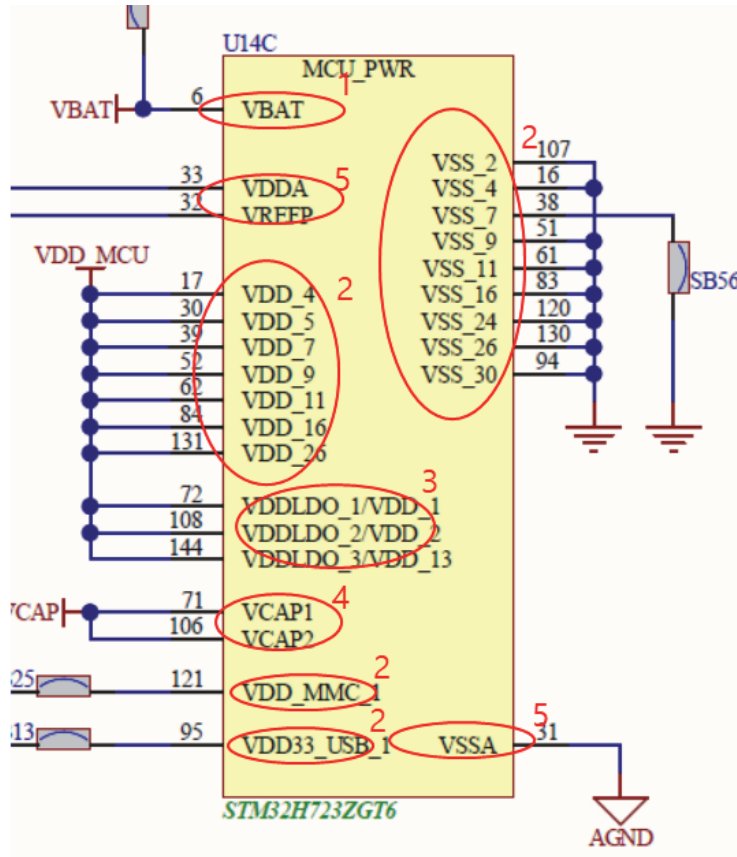
(2) 제작된 PCB를 테스트하고 신호 품질, 안정성 및 기능을 확인한다.

(3) 테스트 결과를 분석하여 필요한 수정 사항을 정리하여 수정 설계에 반영한다.

근래 손쉬운 접근 방법은 이미 상품화된 보드를 검색하거나, 상품 보드 제조 업체에 자문을 구하여 기성품을 기반으로 요구 사양을 반영한 수정 과정을 거쳐 회로를 제작하는 것이다. 이때 기성품과 요구 사양을 비교하여 수정 개발을 최소화하는 방법을 선택하는 편이 가격, 품질 기간 측면이나 수급 안정성 측면에서 유리하다.

② MCU 전원 회로 설계 가이드

MCU에 전원 회로 부분을 설계할 때 전원 관련 핀이 단순하지 않으므로 각 핀의 용도를 확인하고 잘 매치시켜야 한다. 예를 들어 STM32H723ZGT6의 회로를 아래 [그림 1-1]을 참고하며 살펴본다.



출처: 집필진 제작(2023)

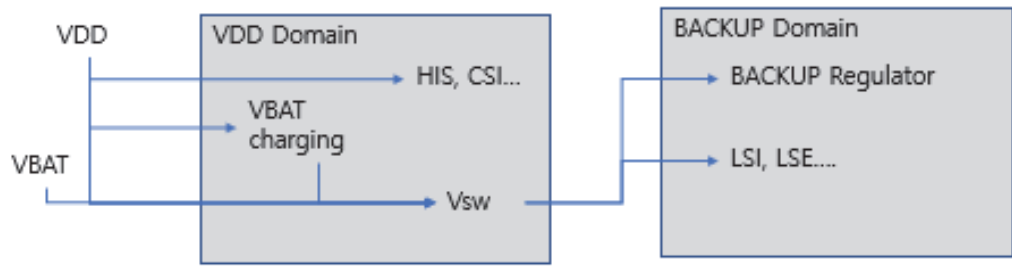
[그림 1-1] MCU 보드 전원 회로도 사례

1. VBAT

- (1) 배터리를 연결하는 포트이다.
- (2) 주전원(VDD)이 공급되지 않을 때에 VBAT 전원으로 RTC, RTC 레지스터 및 백업 SRAM에 전원을 공급하여 동작시킨다.
- (3) 추가로 저전력 모드와 관련된 내용은 심화 학습 과정에서 연구해 본다.

2. VDD/VSS

- (1) 포트에 전원을 공급한다.
 - (가) VDD는 VSS와 매칭되어야 한다.
 - (나) VDD에 +1.8V~3.3V를 연결하고 VSS에 GND를 연결한다.

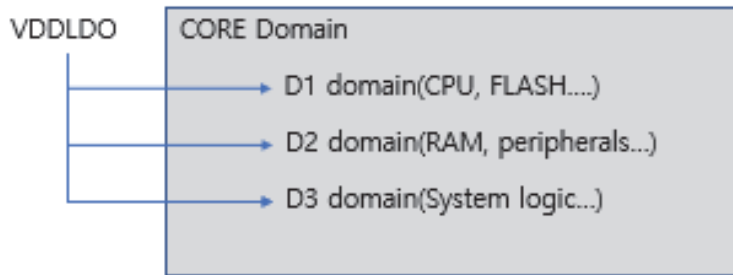


출처: 집필진 제작(2023)

[그림 1-2] VBAT과 VDD 설명(약식 설명이므로 정확한 회로는 참고 자료 참조)

3. VDDLDO_n/VDD_n

MUC core에 전원 공급을 담당하는 LDO(전압조정기)에 전원을 공급한다.



출처: 집필진 제작(2023)

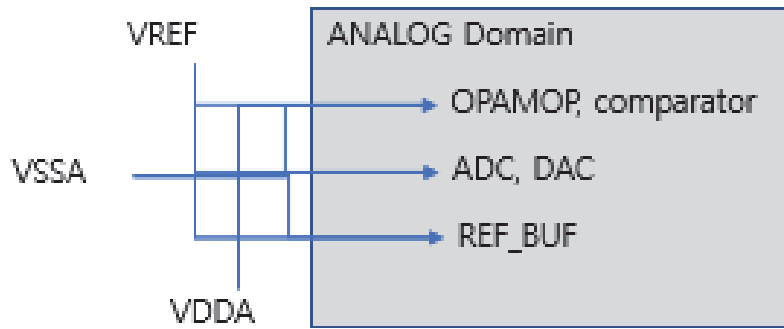
[그림 1-3] VDDLDO 설명(약식 설명이므로 정확한 회로는 참고 자료 참조)

4. VCAP

- VCAP은 내부 LDO 출력과 연결한다.
- MCU 외부 GND에 연결한다(데이터 시트 참고).

5. VDDA/VSSA/VREF

VDDA, VSSA, VREF는 MCU 기능 중에 아날로그 신호를 처리하는 AD Converter, DA Converter에 전원을 공급한다.



출처: 집필진 제작(2023)

[그림 1-4] VDDA, VSSA, VREF 설명(약식 설명이므로 정확한 회로는 참고 자료 참조)

MCU 회로 정보 검색

회로도와 데이터 시트를 참조하여 보면 각 핀에 연결해야 하는 커패시터의 값과 수량이 나온다. 모든 자료는 제조사 문서를 검색하면 찾을 수 있지만 본 예제의 경우에는 getting start with “stm32h” mcu hardware development의 타이틀로 검색하여 에서 따옴표 안에 원하는 제품명을 넣고 검색하면 쉽게 찾을 수 있다. 다음을 참고한다.

<https://pdf4pro.com/view/getting-started-with-stm32-mcu-hardware-development-353de4.html>

https://www.st.com/resource/en/application_note/an5419-getting-started-with-stm32h723733-stm32h725735-and-stm32h730-value-line-hardware-development-stmicroelectronics.pdf

③ 데이터 타이밍 인터페이스

1. 정의

데이터 타이밍 인터페이스는 디지털 시스템에서 데이터 전송의 타이밍과 관련된 신호와 규칙을 정의하는 인터페이스를 말한다.

2. 역할

데이터 타이밍 인터페이스는 데이터 전송 시에 언제 데이터를 보내고 받아야 하는지를 정확하게 정의하고, 이를 기반으로 시스템의 동기화와 데이터의 정확한 전달을 보장한다.

3. 신호와 규칙

데이터 타이밍 인터페이스는 여러 가지 신호와 규칙으로 구성된다. 일반적으로 사용되는 몇 가지 데이터 타이밍 신호와 규칙에는 다음과 같은 것들이 있다:

(1) 클럭(Clock)

(가) 클럭 신호는 시스템의 기본 타이밍 신호로 사용된다.

(나) 클럭은 일정한 주기로 변화하며, 이 주기에 맞추어 데이터가 전송되고 처리된다.

(다) 데이터의 변화는 보통 클럭의 상승 또는 하강 에지에 맞춰서 이루어진다.

(2) 데이터 비트(Data Bit)

(가) 데이터 비트는 전송되는 실제 데이터를 나타낸다.

(나) 데이터 비트는 클럭 신호의 에지에 따라 전송되며, 보통 여러 비트로 구성된 데이터 버스를 통해 전송된다.

(3) 데이터 유효 시간(Data Valid Time)

(가) 데이터 유효 시간은 데이터 비트가 유효한 시간 동안에만 읽혀야 한다는 것을 나타낸다.

(나) 데이터 유효 시간은 클럭 신호와 함께 사용되어 데이터의 올바른 타이밍을 보장한다.

(4) 지연 시간(Delay Time)

(가) 지연 시간은 데이터가 전송 소스에서 전송 목적지까지 이동하는 데 걸리는 시간을 나타낸다.

(나) 지연 시간은 데이터 타이밍을 계획하고 동기화하는 데 중요한 요소이다. 이러한 데이터 타이밍 인터페이스는 디지털 시스템에서 데이터의 신뢰성과 정확성을 보장하기 위해 중요한 역할을 한다. 데이터가 정확한 타이밍과 순서에 따라 전송되면 시스템의 동작이 올바르게 이루어지며, 데이터의 손실이나 오류를 방지할 수 있다. 그러므로 데이터 타이밍 인터페이스의 설계와 구현은 신뢰성 있는 디지털 시스템의 핵심 부분 중 하나이다.

4 적층과 부품 밀도

1. 회로의 부품 밀도

(1) 회로의 부품 밀도는 회로에 사용되는 부품의 개수가 단위 면적에 얼마나 밀집되어 있는지를 나타내며 이는 회로의 복잡성과 밀도를 나타내는 지표로 사용된다.

(2) 부품 밀도가 높다는 것은 주어진 공간에 많은 부품이 밀집되어 있다는 의미이다.

2. PCB의 적층

PCB의 적층은 PCB(Printed Circuit Board)이라는 회로 기판을 여러 개의 층으로 구성하는 기술을 말한다. PCB의 적층은 단층 PCB보다 더 많은 신호 경로를 구성할 수 있고, 회로의 밀도를 증가시킬 수 있다. PCB 적층은 회로의 복잡성이 높아도 공간을 효율적으로 활용할 수 있게 해주는 장점을 가지고 있다.

3. 부품 밀도와 적층

두 개념 사이에는 밀접한 관련성이 있다. 회로의 부품 밀도가 높을수록 PCB의 적층이 유리한 선택일 수 있다. 부품이 밀집되면 신호 경로가 짧아지고 회로의 복잡성이 증가하게 된

다. 이러한 상황에서 PCB의 적층을 통해 회로의 복잡성을 처리하고 추가적인 신호 경로를 분리하거나 구성할 수 있으며, 회로의 밀도를 향상시키고, 노이즈를 차단하는 등의 기능을 수행할 수 있다. 따라서 부품 밀도와 PCB의 적층은 서로 비례적으로 연관되며 회로 설계와 PCB 레이아웃 과정에서 고려되어야 하는 중요한 요소이다.

⑤ 부품의 배치와 회로의 구성 대비 신호의 간섭과 노이즈

부품의 배치와 회로의 구성이 신호의 간섭과 노이즈에 미치는 영향을 고려하면 다음과 같은 주의사항을 게을리할 수 없다.

1. 물리적 분리

노이즈와 간섭을 최소화하기 위해 민감한 신호 경로와 노이즈 발생원을 물리적으로 분리하는 것이 중요하다. 가능한 경우 신호 경로와 노이즈 발생원 사이에 충분한 간격을 유지하고, 각 부품을 적절한 위치에 배치하여 상호 간섭을 최소화해야 한다.

2. 접지(Grounding)

접지는 회로 설계에서 중요한 요소이다. 적절한 접지 방식을 선택하고 구현해야 한다. 접지선의 길이를 최소화하고, 디지털과 아날로그 회로의 접지를 분리하거나 격리하는 등의 접지 전략을 고려해야 한다.

3. 신호 경로의 길이와 미터링

신호 경로의 길이가 길어질수록 신호의 간섭과 노이즈 문제가 발생할 수 있다. 가능한 한 신호 경로를 짧게 유지하고, 미터링을 통해 신호의 품질을 모니터링하고 개선할 수 있도록 해야 한다.

4. 적절한 필터링

필요에 따라 필터링 회로를 추가하여 노이즈를 제거하거나 감소시킬 수 있다. 예를 들어, 파워 라인에 필터 콘덴서를 추가하여 고주파 노이즈를 제거하거나, 신호선에 필터 인덕터를 사용하여 고주파 간섭을 차단할 수 있다.

5. 신호선 간격과 경로

신호선 간의 충분한 간격을 유지하여 신호 간 간섭을 방지할 수 있다. 또한, 신호선의 경로를 가능한 한 민감한 회로와 격리하는 것이 좋다.

이러한 규칙과 주의사항을 고려하면 MCU부 회로의 신호 간섭과 노이즈 문제를 최소화하고 신호의 신뢰성과 정확성을 향상시킬 수 있다. 그러나 회로 설계는 복잡하고 다양한 요소를 고려해야 하는 작업이므로 전문적인 지식과 경험이 필요할 수 있다. 따라서 필요한 경우 전문가의 도움을 받는 것이 좋다.

재료·자료

- 블루투스 모듈 HC-06
- USB to UART 변환장치(Cp2102)
- 저항, LED 단자, 커패시터
- 만능 PCB
- CMOS IC
- TTL IC
- 와이어

기기(장비 · 공구)

- 납땜용 솔더링 아이언 및 솔더
- 납땜 홀더 또는 납땜 스탠드
- 오실로스코프
- 신호 발생기 (Signal generator)
- 오실로스코프

안전 · 유의 사항

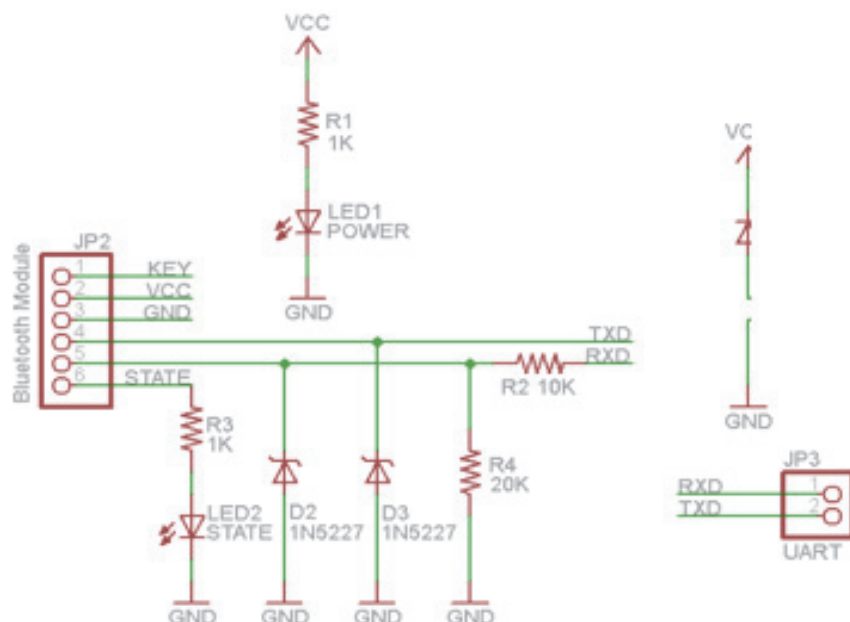
- 전압 레벨 호환성, 전력 소비 관리 및 속도와 딜레이에 관해 사전에 검토한다.
- 노이즈의 영향을 사전에 검토해 본다.
- TTL 신호가 CMOS 회로로 전달될 때 회로에 포함된 에나이어터(EC) 다이오드를 사용하여 CMOS 회로를 보호할 수 있음을 미리 확인하고 필요성을 검토한다.
- 부품의 극성 혼동, 단자 방향 혼동 그리고 보드의 앞뒤를 헷갈리지 않도록 주의한다.

수행 순서

① 로봇의 통신 회로 설계 중 일부인 TTL-CMOS 변환 회로를 제작한다.

1. 보드 설계 및 회로도

MCU에 사용될 UART에는 5V의 전압이 걸리기 때문에 블루투스 모듈이 MCU와의 통신을 위해 통신 전압 레벨의 변환이 필요한 상태이다. 준비한 블루투스 모듈은 HC-06이고 이를 활용하기 위해 레벨 변환 지그를 만든다.



출처: 집필진 제작(2023)

[그림 1-5] TTL-CMOS 레벨 변환 회로 예제

대개 UART 통신을 할 때 5V 레벨의 전압으로 High와 Low로 구분하여 통신하는데 HC-06는 3.3V의 CMOS 레벨로 통신한다. 그러므로 통신선을 AVR과 그대로 연결하면 문제가 생긴다. 이를 위하여 CMOS-TTL 간의 통신 레벨을 변환할 필요가 있어 [그림 1-1]과 같이 회로를 설계했다. 이를 부분별로 요약하면 다음과 같다.

- (1) D1은 실수로 케이블을 반대로 연결했을 시에 전원이 들어오지 않도록 해 둔 안전용 정류다이오드이다.
- (2) 블루투스 모듈의 RXD측은 R2와 R4가 1 : 2로 저항 분압되어 모듈측에 RXD에는 3.3V레벨로 스윙하게 된다. 스윙이란 High-Low 간 변화를 일컫는다.
- (3) D2와 D3는 AVR 컨트롤러 보드에서 입력하는 전원이 최대 6V까지 입력될 수 있고 저항 분

압을 해도 3.8V 정도까지 상승할 우려가 있다. 그러므로 3.3V의 +10% 이내인 3.6V 이상 올라갈 수 없도록 3.6V급 제너다이오드를 연결해 두었다.

(4) LED1, LED2는 각각 LED1은 전원입력상태 확인용, LED2는 블루투스 연결 확인용이다.

2. HC-06 블루투스 모듈 설정

이제 실제 통신에 이용하기 위해서 블루투스 모듈에 기본적인 설정이 필요하다. 설정 내역은 다음과 같다.

- 통신속도 설정 필요
- 블루투스 핀코드 설정
- 블루투스 장치 이름 변경

별도로 준비했던 USB to UART 변환장치와 블루투스 모듈 간에 연결한다.

수행 tip

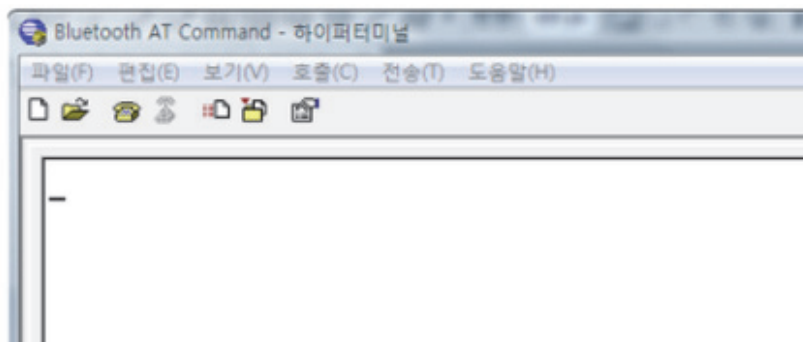
이때 연결은 크로스로 연결해야 한다. 즉.

- UART Device RXD는 Bluetooth TXD에,
- UART Device TXD는 Bluetooth RXD에.

모듈에 따라 다르지만, 일부 종류의 경우 본인을 기준으로 나가는 것이 TX 들어오는 것이 RX로 되어있는 것을 확인해야 한다.

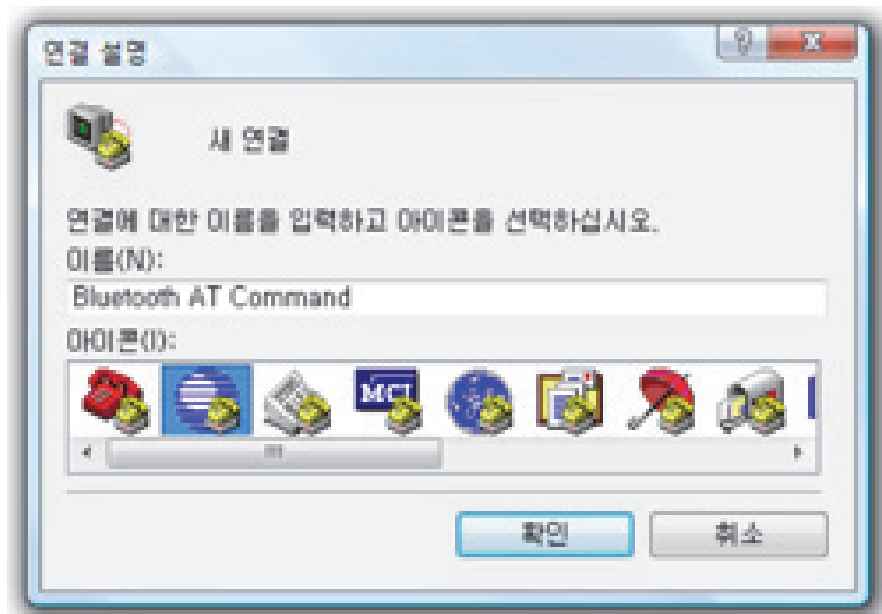
연결을 완료했다면 이제 하이퍼터미널이나 기타 시리얼 통신용 프로그램을 실행하는데 하이퍼터미널을 사용하면 설정 순서는 다음과 같다.

3. 하이퍼터미널 실행



출처: 집필진 제작(2023)
[그림 1-6] 하이퍼터미널 실행

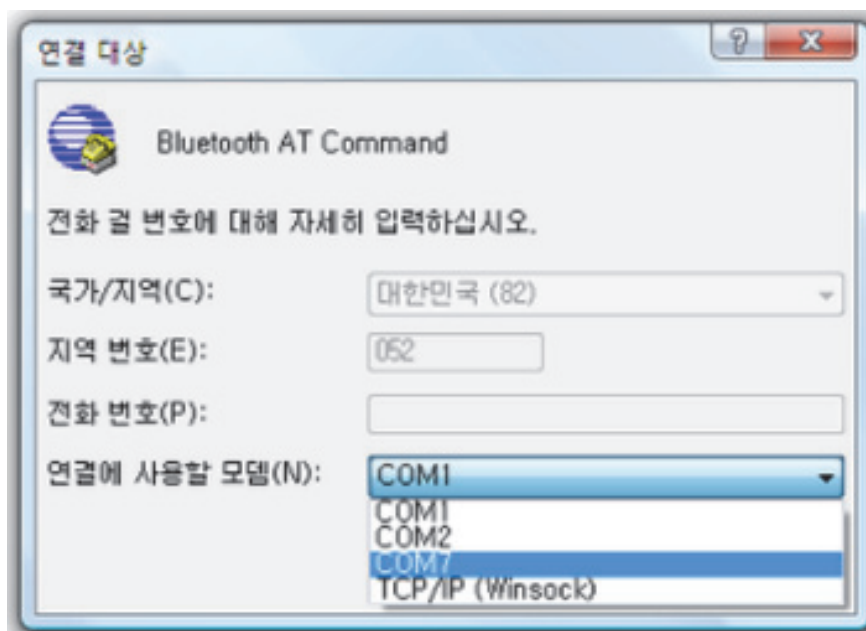
4. 새 연결 설정



출처: 집필진 제작(2023)
[그림 1-7] 새 연결

5. COM 포트 선택

연결에 사용할 모뎀을 USB to UART로 변환된 COM 포트는 컴퓨터마다 숫자가 다를 수 있다. 이는 장치관리자의 포트에서 확인할 수 있다.



출처: 집필진 제작(2023)
[그림 1-8] COM 포트 선택

6. 포트설정 변경

통신방식/속도에 맞게 포트 설정을 변경하는데 HC-06의 공장 초기 통신속도는 9600 bps 이고, 데이터비트는 8비트, 스톱비트 1비트, 흐름제어는 없음임을 확인한다.



출처: 집필진 제작(2023)
[그림 1-9] 포트 설정 변경

이제 통신 설정을 마쳤다.

7. 통신테스트인 AT 명령 시도

하이퍼터미널의 속성에서 입력되는 텍스트를 화면에 표시하는 옵션을 체크해 두면 화면에 문자가 입력된다. AT 명령을 보냈을 때 OK가 수신된다면 설정이 가능한 상태입니다. 이제 원하는 설정값을 AT 명령어로 보내준다. 이는 메모장 등에서 AT명령을 복사하는 것이다. 예를 들어 스마트폰에서 블루투스를 검색했을 때 내 장치가 Bluetooth로 검색되도록 하고 싶다면 AT+NAMEBluetooth로 명령을 날려주면 된다. 아래 [그림 1-7]을 참고한다.

1. Baud rate 변경

Sent : AT+BAUD1

receive : OK1200

Sent : AT+BAUD2

receive : OK2400

- baud rate 값

1-----1200

2-----2400

3-----4800

4-----9600

5-----19200

6-----38400

7-----57600

8-----115200

- Baud rate 설정은 전원 OFF시 저장할수 있다.

2. 블루투스 이름 변경

Sent : AT+NAMEdevicename

receive : OKname

- 디바이스 이름은 사용자가 정하는데 디바이스 이름으로 검색되어 진다.

- 디바이스 이름 설정은 전원 OFF시 저장할 수 있습니다.

3. Pincode 변경

Sent : AT+PINxxxx

receive : OKsetpin

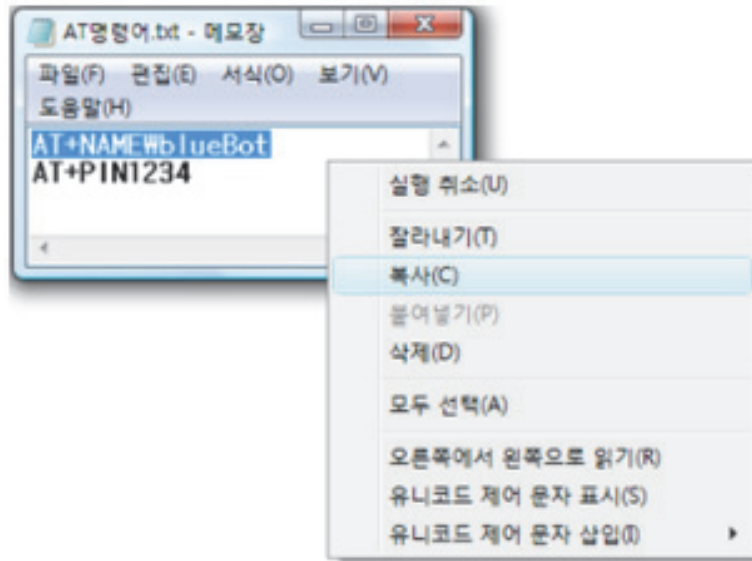
- xxxx 는 설정되는 pin code 값이다.

- PIN code 설정은 전원 OFF시 저장할수 있습니다.

출처: 집필진 제작(2023)

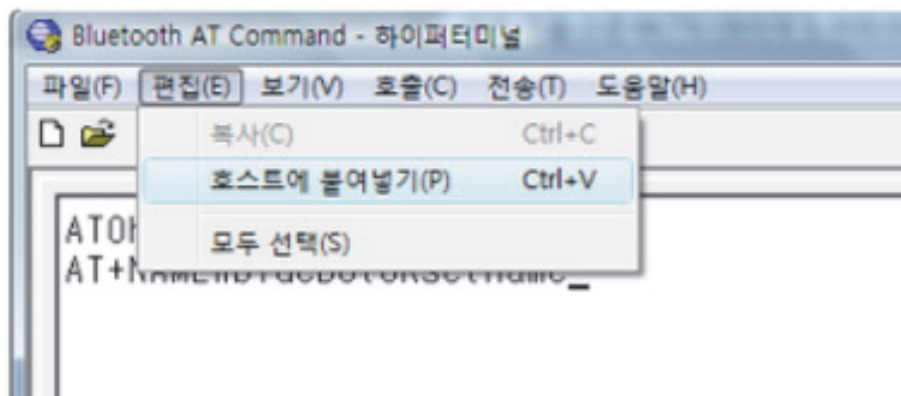
[그림 1-10] HC-06에서 지원하는 3가지 명령

메모장에 입력해둔 AT 명령을 복사한다.



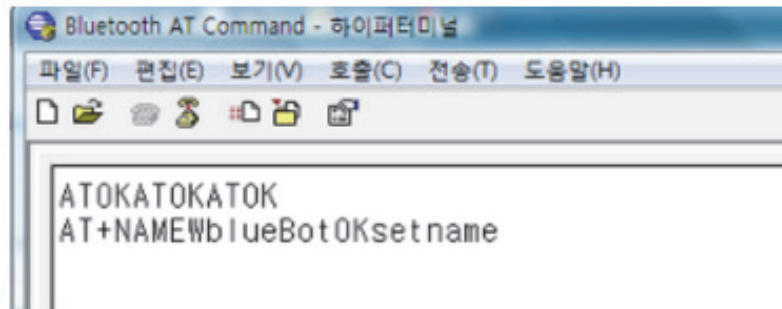
출처: 집필진 제작(2023)
[그림 1-11] AT 명령 복사

8. 하이퍼터미널에 명령 붙여넣기



출처: 집필진 제작(2023)
[그림 1-12] 하이퍼터미널에 명령어 붙여 넣기 1

이제 원하는 설정값을 AT 명령어로 보내는데, 명령을 보냈을 때 OK 혹은 OKsetname, OKsetpin, OK통신속도 등으로 수신이 된다면 설정이 정상적으로 등록된 상태이다. 그러면 다른 AT 명령도 다음과 같이 설정해 준다.



출처: 집필진 제작(2023)

[그림 1-13] 하이퍼터미널에 명령어 붙여 넣기 2

설정을 마친 스마트폰으로 블루투스 검색을 해서 제대로 되었는지 확인해 본다.

9. 컨트롤러 보드 설계 및 회로도

- (1) [그림 1-14]의 회로에서 중요하게 한가지 변한 것이 있다면 크리스탈이다. UART 통신을 할 때 보오레이트 주기를 설정하는데 그때 크리스탈값과 속도에 따라 에러율이 있다. 8Mhz 크리스탈의 경우 9600 bps의 전송속도 사용 시에 0.2%의 에러율이 있어서 크게 문제없으나, 시험 삼아 7.3728Mhz 크리스탈을 사용해서 에러율을 0%로 맞췄다.
- (2) 이 회로는 블루투스 변환 보드와 연결 할 수 있도록 RX, TX 통신 단자에 핀 헤더를 연결해서 UART 통신커넥터를 만들어두고, 블루투스 모듈에 전원을 공급할 수 있도록 따로 POWER 단자도 만들어 두었다.
- (3) 포트 D에 2, 3번 핀을 핀 헤더로 2핀 빼두어서 확장 I/O 포트도 만들었다. 향후 확장성을 생각해 볼 때 간단히 LED를 붙일 수도 있고 다른 활용도 가능하다.

CMOS-TTL 변환

1. CMOS와 TTL의 비교

〈표 1-1〉 CMOS와 TTL 비교

CMOS(Complementary Metal Oxide Semiconductor)	TTL(transistor transistor logic)
- CMOS 로직은 쌍대 MOSFET (n-type과 p-type)을 사용하여 구현된다. - CMOS 회로는 정지 상태에서는 거의 전력을 소비하지 않는다. 이는 CMOS 회로가 전력을 소비하는 시점이 전환 (상승 또는 하강)에서만 발생하기 때문이다. - CMOS 로직은 입력 임피던스가 매우 높아 (즉, 입력이 회로에 거의 영향을 미치지 않음) 입력 전력이 매우 낮다. - CMOS 로직은 노이즈 마진이 크므로 더 안정적이다. - CMOS 로직은 보통 더 높은 집적도를 가진다. 이는 보다 더 많은 기능을 동일한 공간에 넣을 수 있음을 의미한다.	- TTL 로직은 바이폴라 접점 트랜지스터를 사용하여 구현된다. - TTL 회로는 전력 소비가 CMOS보다 높다. - TTL 로직은 입력 임피던스가 낮고 출력 임피던스가 높다. - TTL 로직은 보통 빠른 전환 속도를 가진다.

2. 변환

CMOS-TTL 간의 변환은 두 로직 패밀리 간에 전력 소비, 속도, 전압 수준 등 다양한 특성이 다르기 때문에 변환 작업이 필요할 때 사용한다. 그 상황은 다음과 같다.

- (1) 전력 호환성: CMOS와 TTL 로직은 전원 공급 방식 및 전압 수준이 다르다. 따라서 시스템에서 전원을 공급하는 방식에 따라, CMOS에서 TTL로 또는 TTL에서 CMOS로의 변환을 필요로 할 수 있다.
- (2) 시스템 호환성: 다른 로직 패밀리를 사용하는 기존 시스템과의 호환성을 유지하기 위해 변환이 필요할 수 있다. 예를 들어, CMOS 기반의 새로운 로직을 TTL 기반 시스템에 통합하려면 CMOS 신호를 TTL 수준으로 변환해야 한다.
- (3) 신호 강도: CMOS와 TTL은 출력 신호의 강도가 다르다. CMOS는 높은 임피던스 출력을 제공하며, TTL은 저항적인 출력을 하고 있다. 따라서 출력 신호의 강도를 맞추기 위해 변환이 필요할 수 있다.

변환 회로는 이러한 호환성 및 특성 차이를 고려하여 디지털 신호의 수준을 적절하게 변환하는 역할을 한다. 일반적으로 CMOS-TTL 변환은 레벨 시프터(IC 또는 전송 게이트)를 사용하여 수행된다. 이를 통해 서로 다른 로직 패밀리 간에 신호 호환성을 유지하면서 시스템을 통합할 수 있다.

수행 tip

- 설계에 참고하기 위해 PCB(인쇄 회로 기판) 회로도 및 아트웍의 예시 사진이나 그림을 찾아야 한다면 참고하기 좋은 몇 가지 리소스가 있다.

1. 온라인 전자 커뮤니티

전자 및 PCB 설계 전용 웹사이트와 포럼에는 사용자가 프로젝트와 설계를 공유하는 섹션이 있는 경우가 많다. 일렉트로닉스 스택 익스체인지(electronics.stackexchange.com)는 그중 인기 있는 커뮤니티이고 그와 비슷한 EEV블로그 포럼(www.eevblog.com/forum), 레딧(<https://www.reddit.com/r/AskElectronics/>) 또는 GitHub(<https://github.com/>), Hackaday(<https://hackaday.com/>)와 같은 웹사이트에는 회로도와 PCB 디자인을 찾을 수 있는 사용자 기여 프로젝트와 리포지토리가 있는 경우가 많다. 프로젝트 섹션을 둘러보거나 특정 키워드를 입력하여 관련 예제를 검색할 수 있다.

2. PCB 설계 소프트웨어 웹사이트:

많은 PCB 설계 소프트웨어 제공업체가 웹사이트에서 예제 프로젝트와 디자인을 제시한다. 인기 있는 PCB 설계 소프트웨어 회사로는 Altium(<https://www.altium.com/kr/>), Eagle(<https://www.sparkfun.com/EAGLE>), KiCad(<https://www.kicad.org/>), OrCAD(<https://www.orcad.com/0>) 등이 있다. 해당 웹사이트 또는 커뮤니티 포럼을 방문하면 예제 디자인에 액세스할 수 있다.

3. 온라인 리포지토리

GitHub(github.com) 및 GitLab(gitlab.com)과 같은 웹사이트에는 수많은 오픈 소스 하드웨어 프로젝트가 있다. 이러한 플랫폼에서 PCB 프로젝트를 검색하고 커뮤니티에서 공유한 회로도 및 아트웍을 찾을 수 있다.

4. 온라인 튜토리얼 및 강좌

Udemy(www.udemy.com), Instructables(<https://www.instructables.com/>), Hackster(<https://www.hackster.io/>) 및 Coursera(www.coursera.org)와 같은 온라인 학습 플랫폼에서 PCB 설계에 관한 강자를 제공한다. 이러한 강좌에는 액세스할 수 있는 샘플 프로젝트와 리소스가 포함된 경우가 많다.

5. 제조업체 웹사이트

일부 전자 부품 제조업체 및 공급업체는 웹사이트에서 참조 설계 및 애플리케이션 노트를 제공한다. 텍사스 인스트루먼트(<https://www.ti.com/>), 마이크로칩(<http://www.microchipkorea.com/html/main/main.asp>), 아날로그 디바이스(<https://www.analog.com/en/landing-pages/>)

001/korea-welcome.html)나 STM32(<https://www.st.com/ko/stm32/stm32/stm32intro.html>) 같은 회사는 종종 자사 제품의 예시로 PCB 설계 및 회로도를 제공한다.

6. 오픈 소스 하드웨어 프로젝트

Arduino(<https://www.arduino.cc/>), 라즈베리파이(<https://www.raspberrypi.org/>), 아다프루트(<https://io.adafruit.com/>) 같은 플랫폼은 오픈 소스 하드웨어 설계를 제공한다. 해당 저장소 또는 프로젝트 갤러리를 탐색하여 예제 회로도 및 PCB 설계를 찾을 수 있다.

7. PCB 설계 블로그 및 튜토리얼

많은 전자제품 애호가와 전문가가 블로그와 튜토리얼을 통해 지식과 프로젝트를 공유한다. SparkFun(<https://www.sparkfun.com/>)나 CircuitDigest(<https://circuitdigest.com/>)와 같은 웹사이트는 종종 회로도 및 PCB 레이아웃이 포함된 단계별 가이드를 제공하여 학습하고 예제로 사용할 수 있다.

이때 주의할 것은 외부 소스에서 얻은 예제 설계 또는 회로도를 사용할 때는 파일과 관련된 라이선스 또는 사용 제한 사항을 필히 준수해야 한다는 것이다. 그리고 필요한 경우 원본 제작자에게 적절한 보상이나 대가를 제공할 수도 있다. 예제 디자인을 사용할 때는 지식재산권 및 라이선스를 존중해야 하므로 항상 필요한 권한이 있는지 확인하고 해당 사용 지침을 따라야 한다.

수행 내용 2 / MCU 회로 설계 시 오류별로 예방 대책 수립하기

재료·자료

- MCU 회로도
- MCU 하드웨어 예제
- UART, I2C, SPI 등의 시리얼 통신 인터페이스 (Serial communication interfaces like UART, I2C, SPI)
- GPIO (General Purpose Input/Output) 핀 및 인터페이스

기기(장비 · 공구)

- 전원 공급 장치 (Power supply unit)
- 배터리 또는 충전기 (Battery or charger)
- 전원 관리 모듈 (Power management module)
- 전압 레귤레이터 (Voltage regulator)
- 전압 변환기 (Voltage converter)
- 전압 모니터 (Voltage monitor)
- 디지털-아날로그 변환기 (Digital-to-analog converter, DAC)
- 아날로그-디지털 변환기 (Analog-to-digital converter, ADC)
- 전력 측정 장치 (Power measurement device)
- 전력 모니터 (Power monitor)
- 전력 소비 분석기 (Power consumption analyzer)
- 스펙트럼 분석기 (Spectrum analyzer)
- 노이즈 필터 (Noise filter)
- 지터 분석기 (Jitter analyzer)

- 지연 측정 장치 (Delay measurement device)
- EMC 테스트 장비 (Electromagnetic Compatibility testing equipment)
- 컴퓨터
- 인터넷

안전 · 유의 사항

- 오류 사례를 수집할 때는 신뢰할 수 있는 출처에서 가져온 정보를 사용해야 한다. 신뢰성 있는 웹사이트, 전문적인 기술 문서, 신뢰할 만한 출판물 등을 참고하는 것이 좋다.
- 가능한 한 다양한 원천에서 오류 사례를 수집하는 것이 중요하다. 여러 다른 프로젝트, 포럼, 커뮤니티, 블로그 등을 참고하여 다양한 관점과 경험을 얻을 수 있다.
- 오류의 원인을 분석하고 해당 원인에 관한 이해를 돕는 자료를 검색한다.
- 오류 확인 시에는 해결책을 모색해 보고 검토하여 문제 해결 능력을 스스로 양성한다.
- 수집한 오류 사례를 자신의 회로 설계에 적용할 때는 해당 오류가 실제로 해당하는 상황에 적용 가능한지 평가해야 한다.

수행 순서

① 로봇의 MCU 보드에 주변 장치를 배치할 때의 유의 사항을 알아본다.

로봇의 MCU 보드에 주변 장치를 배치할 때는 몇 가지 규칙과 주의사항을 고려해야 한다.

주요 고려 항목은 다음과 같다.

- 전원 공급
- 전압 레벨
- 신호 인터페이스
- 전력 소비
- 노이즈와 간섭
- 크기와 물리적 제약 사항

1. 전원 공급

(1) 주의점

주변 장치들은 안정적이고 충분한 전원을 공급받아야 한다.

(가) MCU 보드에는 전원 공급을 위한 핀 또는 연결지점이 제공된다.

(나) 주 부품을 연결할 때 해당 전원 핀 또는 연결지점에 올바른 전압과 전류를 공급하는지 확인해야 한다.

(다) 충분한 전압과 전류를 공급하지 않으면 주 부품이 제대로 동작하지 않거나 예기치 않은 동작을 할 수 있다. 이에 따라 시스템의 안정성과 신뢰성에 영향을 줄 수 있다.

(2) 설계 오류 사례

(가) 전원 공급 회로 용량 부족

1) 예시

로봇 시스템에 필요한 전류를 충분히 공급하지 못하여 시스템이 동작하지 않거나 정상적인 성능을 발휘하지 못하는 경우를 예로 들 수 있다.

2) 원인

전원 공급 회로의 저항이나 컨덴서의 용량이 부족하여 전압 강하나 전류 제한이 발생하는 경우 등이다.

3) 해결 방법

전원 공급 회로의 저항을 낮추거나 컨덴서 용량을 증가시켜 충분한 전류 공급이 가능하도록 조치한다.

(나) 전원 공급 회로 접점 문제

1) 예시

전원 공급 회로의 접점에 부식이나 불량 연결 등이 발생하여 전압이 간헐적으로 떨어지거나 전류가 불안정한 경우를 예로 들 수 있다.

2) 원인

전원 공급 회로의 접점이 제대로 연결되지 않거나 부식으로 인해 전압이 불안정한 경우 등이다.

3) 해결 방법

접점을 정확하게 연결하고 청소하여 전압 안정성을 유지하도록 유지보수 및 점검을 수행한다.

(3) 시사점

위의 사례는 전원 공급 회로 설계의 실수로 인해 충분한 전압과 전류가 공급되지 않는 문제를 보여준다. 이러한 문제를 방지하기 위해서는 전원 공급 회로 설계를 신중하게 수행하고, 적절한 저항, 컨덴서, 전선 굵기 등을 선택하여 요구되는 전압과 전류를 안정적으로 공급할 수 있도록 해야 한다.

2. 전압 레벨

(1) 주의사항

- (가) MCU 보드와 주 부품 간에 전압 레벨이 호환되는지 확인해야 한다.
- (나) 대부분의 MCU 보드는 3.3V 또는 5V의 전압 레벨을 사용하므로, 주 부품의 전압 요구 사항을 확인하고 필요한 레벨 변환 회로를 추가해야 할 수도 있다.
- (다) 호환되지 않는 전압 레벨을 사용하면 주 부품이 손상되거나 정상적으로 동작하지 않을 수 있다.
- (라) 전압 레벨 변환 회로를 추가하지 않거나 올바르게 구현하지 않으면 통신 오류 또는 손상된 주변 장치가 발생할 수 있다.

(2) 설계오류 사례

(가) 전압 레벨 설정 오류

1) 예시

보드에 필요한 전압 레벨을 정확하게 설정하지 못하여 회로가 동작하지 않거나 안정적인 신호를 전달하지 못하는 경우를 예로 들 수 있다.

2) 원인

전압 레벨이 회로의 동작 조건과 불일치하거나 설정이 잘못되었을 때 등이다.

3) 해결 방법

전압 레벨을 시스템 요구 사항에 맞게 정확하게 설정하고, 적절한 전압 레벨 변환 회로를 구성하여 신호를 안정적으로 전달할 수 있도록 조치한다.

(나) 전압 강하(드롭) 문제

1) 예시

전압 공급 회로의 저항이나 전원선의 굵기가 적절하지 않아 전압 드롭이 발생하여 시스템 동작이 불안정하거나 전원 요구를 충족하지 못하는 경우를 예로 들 수 있다.

2) 원인

전압 공급 회로의 저항이 크거나 전원선의 굵기가 작아서 전압이 드롭되는 경우 등이다.

3) 해결 방법

전압 공급 회로의 저항을 감소시키거나 전원선의 굵기를 증가시켜 전압 드롭을 최소화하여 충분한 전압 공급이 가능하도록 조치한다.

(3) 시사점

위의 사례는 전압 레벨 설계의 실수로 인해 충분한 전압과 전류가 공급되지 않는 문제를 보여준다. 이러한 문제를 방지하기 위해서는 전압 요구 사항을 정확하게 이해하고,

회로 설계 시 적절한 전압 레벨을 설정하고 전압 드랍을 최소화할 수 있는 회로 및 전원 공급 구성을 선택하여 설계해야 한다. 또한, 충분한 전압 공급을 위해 전원 회로의 저항, 굽기, 전압 레벨 변환 회로 등을 고려해야 한다.

3. 신호 인터페이스

(1) 주의사항

(가) 주 부품과 MCU 보드 사이의 신호 인터페이스를 고려해야 한다. 대부분의 주 부품은 UART, I2C, SPI 등의 표준 프로토콜을 지원한다. MCU 보드에 해당하는 인터페이스를 제공하며, 주 부품과 MCU 보드 사이의 신호선을 올바르게 연결해야 한다.

(나) 올바른 신호 인터페이스를 사용하지 않으면 주 부품과 MCU 간의 통신이 제대로 이루어지지 않을 수 있다. 데이터 전송 오류, 잘못된 읽기/쓰기, 불안정한 통신 등의 문제가 발생할 수 있다.

(2) 설계 오류 사례

(가) 신호 레벨 불일치

1) 예시

신호 송수신을 위한 인터페이스의 신호 레벨이 일치하지 않아 신호가 올바르게 전달되지 않거나 잘못된 동작을 일으키는 경우를 예로 들 수 있다.

2) 원인

송신 측과 수신 측의 신호 레벨이 일치하지 않는 경우 등이다.

3) 해결 방법

신호 레벨 변환 회로를 사용하여 송신 신호를 적절한 레벨로 변환하고 수신 측에서 올바르게 해석할 수 있도록 조치한다.

(나) 인터페이스 통신 속도 불일치

1) 예시

인터페이스 간 통신 속도가 일치하지 않아 데이터 전송이 불안정하거나 데이터 손실이 발생하는 경우를 예로 들 수 있다.

2) 원인

송신 측과 수신 측의 통신 속도가 일치하지 않거나 충분한 대역폭을 가지지 못하는 경우 등이다.

3) 해결 방법

통신 속도를 일치시키기 위해 송신 측과 수신 측을 동일한 속도로 설정하고, 충분한 대역폭을 제공하기 위해 적절한 통신 인터페이스를 선택한다.

(다) 신호선 간 간섭 문제

1) 예시

신호선 사이에 간섭이 발생하여 신호의 품질이 저하되거나 잘못된 동작을 유발하는 경우를 예로 들 수 있다.

2) 원인

신호선이 서로 근접하여 간섭을 발생시키는 경우 등이다.

3) 해결 방법

간섭을 방지하기 위해 적절한 신호 라우팅과 신호선 간 충분한 간격을 유지하고, 필요한 경우 그라운드 평면을 사용하여 간섭을 최소화한다.

(3) 시사점

위의 사례는 신호 인터페이스 설계의 실수로 인해 충분한 전압과 전류가 공급되지 않는 문제를 보여준다. 이러한 문제를 방지하기 위해서는 인터페이스의 신호 레벨을 정확하게 일치시키고, 통신 속도와 대역폭을 적절하게 설정하며, 간섭을 최소화하기 위한 설계 및 신호 라우팅을 신중하게 고려해야 한다.

4. 전력 소비

(1) 주의사항

(가) 주 부품이나 외부 장치가 전력을 많이 소비할 경우, MCU 보드의 전원 공급 능력을 고려해야 한다. 전원 공급 회로가 주 부품의 전력 요구를 충분히 지원할 수 있는지 확인해야 한다.

(나) 전력 요구를 충분히 지원하지 못하는 경우 주 부품이 제대로 작동하지 않거나 예상치 못한 동작을 할 수 있다. 전원 공급 회로가 과부하 되어 전압이 불안정해지거나 전력 손실이 발생할 수 있다.

(2) 설계오류 사례

(가) 전력 소비 예측 오류

1) 예시

보드의 전력 소비를 잘못 예측하여 충분한 전압과 전류 공급을 위한 전원 공급 장치를 구성하지 못한 경우를 예로 들 수 있다.

2) 원인

보드의 실제 전력 소비를 정확하게 예측하지 못한 경우 등이다.

3) 해결 방법

보드의 실제 전력 소비를 정확하게 측정하고, 이를 바탕으로 충분한 전압과 전류를 공급할 수 있는 전원 공급 장치를 선택하고 구성한다.

(나) 전원 회로의 용량 부족

1) 예시

전원 회로의 용량이 보드의 전력 요구에 비해 부족하여 충분한 전압과 전류를 공

급하지 못한 경우를 예로 들 수 있다.

2) 원인

전원 회로의 용량이 적절하게 계산되지 않거나 전원 공급 장치의 용량이 제한적일 때 등이다.

3) 해결 방법

전원 회로의 용량을 충분히 계산하여 충분한 전압과 전류를 공급할 수 있도록 설계하고, 필요한 경우 전원 공급 장치를 업그레이드한다.

(다) 전력 관리 기능의 부재

1) 예시

전력 관리 기능이 부족하여 보드의 전력 소비를 최적화하지 못하고, 결과적으로 충분한 전압과 전류를 공급하지 못한 경우를 예로 들 수 있다.

2) 원인

보드에 전력 관리 기능이 제대로 구현되지 않거나 부족한 경우 등이다.

3) 해결 방법

전력 관리 기능을 보완하여 보드의 전력 소비를 최적화하고, 전원 공급에 필요한 충분한 전압과 전류를 제공할 수 있도록 조치합니다.

(3) 시사점

위의 사례는 전력 소비 설계의 실수로 인해 충분한 전압과 전류가 공급되지 않는 문제를 보여준다. 이러한 문제를 방지하기 위해서는 보드의 전력 소비를 정확하게 예측하고 적절한 전원 공급 장치를 선택하며, 전원 회로의 용량을 충분히 계산하여 설계해야 한다. 또한, 전력 관리 기능을 적절히 구현하여 전력 소비를 최적화하고 보드에 충분한 전압과 전류를 공급할 수 있도록 해야 한다.

5. 노이즈와 간섭

(1) 주의사항

(가) 주 부품이나 외부 장치가 MCU 보드에 노이즈나 간섭을 발생시킬 수 있으므로, 신호선이 깨끗하고 안정적인 신호를 전달할 수 있도록 배치와 물리적인 접촉을 고려해야 한다. 필요에 따라 적절한 저항, 콘덴서, 필터링 회로 등을 추가할 수 있다.

(나) 노이즈와 간섭이 적절하게 관리되지 않으면 주변 장치와의 상호작용에 문제가 발생할 수 있다. 신호의 왜곡, 잘못된 데이터 전송, 장치 간 간섭으로 인해 예기치 않은 동작 등의 문제가 발생할 수 있다. 간단한 예시를 다음과 같이 들 수 있다.

1) 신호 간섭 예시

가) 원래 회로: 신호선과 전원선이 근접하여 놓여있고, 그라운드 평면이 부족한 회로를 예로 들 수 있다.

나) 개선된 회로: 신호선과 전원선을 격리하고, 충분한 그라운드 평면을 제공하여

신호 간섭을 최소화한 회로를 예로 들 수 있다.

2) 노이즈 예시

가) 원래 회로: 전원 공급 회로에 충분한 필터링이 없어 전압 변동이 발생하고, 노이즈가 회로에 영향을 준다.

나) 개선된 회로: 필터링 커패시터를 추가하여 전압 스파이크를 완화하고, 노이즈를 제거한 회로이다.

(2) 설계 오류 사례

(가) 전원 라인의 노이즈

1) 예시

전원 라인에 노이즈가 발생하여 보드에 충분한 정확한 전압을 공급하지 못한 경우를 예로 들 수 있다.

2) 원인

전원 라인이 노이즈에 노출되거나 적절한 필터링이 되어있지 않았을 때 등이다.

3) 해결 방법

전원 라인에 노이즈를 제거하기 위한 필터링 회로를 추가하고, 적절한 저항, 커패시터, 인덕터 등을 사용하여 노이즈를 감소시킨다.

(나) 전선 간섭

1) 예시

다른 전선들과의 근접으로 인해 보드에 간섭이 발생하여 충분한 전압과 전류를 공급하지 못한 경우를 예로 들 수 있다.

2) 원인

전선이 충분한 간격을 유지하지 않거나 적절한 차폐가 되어있지 않았을 때 등이다.

3) 해결 방법

전선들을 적절한 간격으로 배치하고, 필요한 경우 전선을 차폐하여 간섭을 최소화한다.

(다) 지지체 간섭

1) 예시

보드를 지지하는 구조물과의 물리적 간섭으로 인해 충분한 전압과 전류를 공급하지 못한 경우를 예로 들 수 있다.

2) 원인

보드를 지지하는 구조물이 충분한 간격을 유지하지 않거나 전기적으로 접지되지 않았을 때 등이다.

3) 해결 방법

보드와 지지체 사이에 충분한 간격을 유지하고, 구조물을 전기적으로 접지하여 간섭을 방지한다.

(3) 시사점

위의 사례는 노이즈와 간섭 설계의 실수로 인해 충분한 전압과 전류가 공급되지 않는 문제를 보여준다. 이러한 문제를 방지하기 위해서는 전원 라인에 노이즈 필터링 회로를 추가하고, 전선들을 적절한 간격으로 배치하고 차폐하는 등의 조치를 취해야 한다. 또한, 보드와 지지체 사이에 충분한 간격을 유지하고 구조물을 전기적으로 접지하여 간섭을 방지한다. 이러한 실수를 피하기 위해서는 전기 설계 규칙을 준수하고, 적절한 노이즈 및 간섭 방지 기술을 적용해야 한다.

(4) 노이즈와 간섭의 문제 추가 사례

(가) 가정

- 1) 로봇의 MCU 보드에 DC 모터를 연결하는 상황이다.
- 2) 아래 예시에서 사용되는 구체적인 사양(저항값, 커패시터 용량 등)은 상황에 따라 다르다. 실제로는 노이즈 및 간섭의 특성과 시스템 요구 사항을 고려하여 적절한 저항, 커패시터, 필터링 회로를 선택하고 설계해야 한다.

(나) 사례

1) 노이즈 문제 해결을 위한 저항 추가

- 가) 예시: 모터가 작동할 때 전원 라인에 노이즈가 발생하는 경우를 예로 들 수 있다.
- 나) 해결 방법: 모터의 전원 라인에 저항을 추가하여 노이즈를 흡수 또는 제어한다.
- 다) 구체적인 사례: (예를 들어) 10옴의 저항을 모터 전원 라인에 직렬로 연결하여 노이즈를 제어한다.

2) 간섭 문제 해결을 위한 커패시터 추가

- 가) 예시: 모터 작동 때문에 발생하는 전압 스파이크로 인해 주변 장치에 간섭이 발생하는 경우를 예로 들 수 있다.
- 나) 해결 방법: 모터 전원 라인에 커패시터를 추가하여 전압 스파이크를 완화한다.
- 다) 구체적인 사례: 예를 들어 100nF의 커패시터를 모터 전원 라인과 지상(GND) 사이에 병렬로 연결하여 전압 스파이크를 완화한다.

3) 필터링 회로를 추가하여 노이즈 및 간섭 제어

- 가) 예시: 모터 작동 때문에 발생하는 노이즈가 주변 센서에 영향을 주는 경우를 예로 들 수 있다.
- 나) 해결 방법: 필터링 회로를 사용하여 노이즈를 제어하고, 주변 센서에 깨끗한 신호를 전달한다.

다) 구체적인 사례: RC 필터 회로를 구성하여 모터 전원 라인과 지상(GND) 사이에 연결한다. 이 회로는 저항과 컨덴서로 구성되어 노이즈를 제어하고 신호를 정화한다.

(다) 시사점

- 1) 일반적으로, 모터와 MCU 간에 발생하는 노이즈 및 간섭 문제는 모터의 작동 방식, 전원 공급의 안정성, 전선 길이와 레이아웃 등 여러 요인에 의해 영향을 받는다. 예를 들어, 고속 회전하는 대용량 모터일 경우 더 많은 노이즈를 발생시킬 수 있으며, 긴 전원선이나 공유되는 전원 회로를 통해 간섭이 전달될 수 있다.
- 2) 따라서, 구체적인 사양과 설계는 시스템의 특성과 요구 사항에 기반하여 결정되어야 한다. 측정, 분석 또는 시뮬레이션을 통해 노이즈 및 간섭의 특성을 파악하고, 이를 기반으로 저항, 컨덴서, 필터링 회로 등을 선택하고 연결해야 한다.
- 3) RC 필터 회로 역시 선택과 설계가 필요한 부분이다. RC 필터는 저항과 컨덴서로 구성되며, 이들의 값을 결정하기 위해서는 원하는 통과 대역폭, 저항과 컨덴서의 상호작용 등을 고려해야 한다. 따라서 아무 저항과 컨덴서를 연결하는 것은 아니며, 시스템 요구 사항과 특성에 맞게 올바른 값을 계산해서 선택해야 한다.

신호의 간섭과 노이즈를 잘 고려한 설계와 그렇지 않은 설계의 차이점은 회로도를 눈으로만 보고 알 수 있을까?

신호의 간섭과 노이즈를 잘 고려한 설계와 그렇지 않은 설계의 차이는 일반적으로 회로도를 눈으로만 보고 쉽게 알아내기는 어렵다. 이러한 설계의 차이는 전기적인 특성과 신호의 품질에 관한 고려 사항으로서, 전문적인 지식과 시뮬레이션 도구를 사용하여 확인되어야 한다. 눈으로 회로도를 살펴봤을 때, 신호의 간섭과 노이즈를 고려한 설계인지 아닌지를 명확하게 판단하기는 어렵지만 몇 가지 힌트나 특징적인 요소를 관찰하여 추정할 수 있을 수도 있다. 예를 들어, 고주파 신호의 간섭을 줄이기 위해 적절한 그라운드 플레인이나 전원 평면이 있는지, 노이즈 필터링을 위한 커패시터가 있는지 등을 확인할 수 있다. 하지만 이러한 특징적인 요소는 설계자의 의도나 회로 설계의 복잡성에 따라 다양할 수 있으므로 절대적인 판단은 아니다. 따라서, 신호의 간섭과 노이즈를 잘 고려한 설계를 확인하기 위해서는 전문적인 전기 설계 지식과 시뮬레이션 도구를 활용하여 회로의 전기적인 특성과 신호의 품질을 분석하는 것이 필요하다.

6. 크기와 물리적 제약 사항

MCU 보드와 주 부품의 크기, 형태, 물리적인 제약 사항을 고려해야 한다. 주 부품이 MCU 보드와 충돌하지 않고 공간상으로 적절하게 배치될 수 있는지 확인해야 한다. 적절한 크기와 물리적 배치를 고려하지 않으면 주 부품이 충돌하거나 부적절한 위치에 배치되어 기능을 제대로 수행할 수 없다. 검토를 소홀히 하면 배선 문제, 접촉 오류, 물리적 손상 등이 발생할 수 있다. 근래에는 이러한 사례는 극히 드문 편이다.

공간이 제한되어 있어서 부품을 배치하고 나서 회로를 설계하는 게 맞을까, 회로 설계를 하고 부품을 배치하는 게 맞을까?

일반적으로는 회로 설계를 먼저 하고, 그 후에 부품을 배치하는 것이 일반적인 절차이다. 이는 회로의 기능, 성능, 신호 경로 등을 고려하여 회로 구조를 설계한 후, 그에 맞게 부품을 배치하는 것이 더 효율적이고 적합하다.

회로 설계를 먼저 진행하면 다음과 같은 이점이 있다.

1. 기능 및 성능 고려

회로 설계를 먼저 하면 기능 요구사항과 성능 목표에 맞게 회로를 설계할 수 있다. 각 기능과 신호 경로를 고려하여 회로의 블록 다이어그램을 작성하고, 이를 기반으로 세부 회로를 설계할 수 있다.

2. 신호 경로 최적화

회로 설계를 먼저 하면 신호의 최적 경로를 고려하여 회로를 구성할 수 있다. 이는 신호의 전달 시간을 최소화하고, 잡음 및 간섭을 최소화하여 회로의 성능을 향상시킬 수 있다.

3. 회로 최적화 및 분석

회로 설계를 먼저 하면 회로를 최적화하고 성능을 분석하는 과정을 보다 체계적으로 수행할 수 있다. 회로 설계 도구 및 시뮬레이션 도구를 사용하여 회로의 동작을 시뮬레이션하고 분석하여 설계의 문제점을 해결하고 최적화할 수 있다.

4. 예외적 경우

한편, 특정한 제약 조건이나 물리적인 요구 사항에 따라 부품의 배치가 중요한 경우에는 일부 부품의 위치를 먼저 결정한 후에 회로 설계를 진행하기도 한다. 이는 공간 제약이나 열 관리 등의 이슈를 고려하여 부품의 배치를 최적화하는 것이다. 그러나 대부분은 회로 설계를 먼저하고, 이를 바탕으로 부품의 배치를 결정하는 것이 더 일반적인 절차이다.

② 신호의 간섭과 노이즈를 고려한 설계의 판별법을 연구한다.

신호의 간섭과 노이즈를 잘 고려하지 못한 설계를 판단하기 위해서는 몇 가지 접근 방법을 사용할 수 있다.

1. 신호 품질 분석

설계된 회로의 시뮬레이션 도구를 사용하여 신호의 품질을 분석한다. 이를 통해 신호의 왜곡 정도, 잡음 레벨, 간섭의 존재 여부 등을 확인할 수 있다. 만약 신호가 왜곡되거나 잡음이 많거나 간섭이 있는 경우, 이는 신호의 간섭과 노이즈를 고려하지 못한 설계일 가능성이 있다.

2. 측정 및 분석

제작된 회로를 실제로 측정하여 신호의 품질을 분석한다. 이를 통해 신호의 왜곡, 잡음, 간섭 등을 확인할 수 있다. 측정 결과가 기대와 다르거나 신호의 품질이 저하되었다면, 신호의 간섭과 노이즈를 고려하지 못한 설계 가능성이 있다.

3. 전기 설계 원칙 검토

설계된 회로의 전기 설계 원칙을 검토한다. 전원 라인의 필터링, 그라운드 플레인의 구성, 신호 라인과 전원 라인의 분리, 각각의 신호 경로에 관한 격리 및 차폐 등을 고려해야 한다. 만약 이러한 원칙이 충분히 고려되지 않았다면, 신호의 간섭과 노이즈를 고려하지 못한 설계일 가능성이 있다.

전자 공학 설계에서 사용되는 회로도에는 전자 설계 도구를 통해 작성되는 것이 일반적이다. 시각적인 도면은 작성 도구에 따라 다를 수 있으며, 구체적인 회로에 따라 달라질 수 있다.

회로 설계 후 검증을 목적으로 한다면, 전자 설계 도구를 사용하여 회로도를 작성하고 확인하는 것이 가장 적합하다. 전자 설계 도구에는 Eagle, Altium Designer, KiCad, LTspice 등 다양한 도구가 있으며, 이러한 도구를 사용하여 회로도를 시각화하고 구체적인 설계를 수행할 수 있다. 이러한 도구는 전자 공학 분야에서 널리 사용되며, 회로의 시각적인 표현과 구성 요소의 상호 연결 등을 제공한다.

이 외에 참고용으로 전자 회로도를 찾을 수 있는 몇 가지 온라인 리소스는 다음과 같다.

- Digi-Key

Digi-Key는 전자 부품 유통 업체로, 다양한 회로도 및 참고 자료를 제공한다. Digi-Key 웹사이트에서 "Design Resources" 섹션을 찾아볼 수 있다.

- Electronics-Lab

Electronics-Lab은 전자 공학 커뮤니티 및 자료 공유 플랫폼으로 회로도, 프로젝트, 튜토리얼 등을 제공하고 있으며, 다른 사용자들과의 상호작용도 가능하다.

- GitHub

GitHub는 오픈 소스 개발 플랫폼으로, 다양한 회로도와 프로젝트가 저장되어 있다. GitHub에서 회로 설계와 관련된 저장소를 검색하거나, 오픈 소스 프로젝트를 검색한다.

- Maker 프로젝트 웹사이트

Arduino, Raspberry Pi, ESP8266 등과 같은 메이커 보드와 관련된 웹사이트에서도 다양한 회로도와 프로젝트를 찾을 수 있다. 예를 들어, Arduino 공식 웹사이트인 Arduino Project Hub에서는 Arduino와 관련된 다양한 프로젝트를 확인할 수 있다.

- 전자 공학 포럼 및 커뮤니티

전자 공학 포럼 및 커뮤니티는 다른 전자 엔지니어들과의 상호작용 및 자료 공유의 장소이다. 예를 들어, Electronics Stack Exchange, Reddit의 r/AskElectronics 등이 유용한 자료 및 회로도를 찾을 수 있는 곳이다.

③ 추가적인 유의 사항을 알아본다.

전술한 6가지 외에 설계 실무에서 흔히 발생하는 오류를 조사해 보면 다음과 같다.

1. 온도 관리

(1) 내용

회로의 작동 온도를 고려해야 한다. 과열은 성능 저하나 심지어 손상을 초래할 수 있다. 온도를 감지하고 온도 제어 장치를 사용하여 안정적인 온도 유지에 신경을 써야 한다.

(2) 사례

(가) 사례 1

- 1) 현상: 고열 발생 부품인 CPU를 적절한 열 관리 없이 사용할 경우, 온도 상승으로 인해 성능 저하나 시스템 오류가 발생한다.
- 2) 대책: 적절한 열 관리를 위해 열 싱크, 팬, 열 관리 소프트웨어 등을 사용하여 온도를 안정적으로 유지해야 한다.

(나) 사례 2

- 1) 현상: 회로 보드가 작은 공간에 적재되어 있고 주변 환경 온도가 높은 경우, 회로 보드의 온도 상승으로 인해 전원 공급 회로 또는 신호 처리 회로의 성능이 감소한다.
- 2) 대책: 이를 해결하기 위해 회로 보드의 온도 상승을 줄이기 위한 열 관리 대책을 시행해야 한다.

2. 기계적 충격과 진동

(1) 내용

로봇과 같이 움직이는 환경에서는 기계적 충격과 진동에 민감할 수 있으니 회로 보호를 위해 충격 흡수 장치나 진동 저감 장치를 사용해야 한다. 부품의 안전한 고정 및 신뢰성을 확보하는 것이 중요하다.

(2) 사례

(가) 사례 1

- 1) 현상: 이동 로봇이 불규칙한 지형에서 작동할 때, 강력한 충격이나 진동이 회로 보드에 전달될 수 있다. 이에 따라 회로 부품의 접속부가 느슨해져 이탈하거나 파손 등이 발생할 수 있으며, 심각한 경우 이에 따라 시스템 동작이 중단되기도 한다.
- 2) 대책: 이를 방지하기 위해 충격 흡수 장치나 진동 완화 장치를 사용하여 회로 보드를 보호해야 한다.

(나) 사례 2

- 1) 운송 중인 로봇 또는 장치의 경우, 급격한 변속, 회전, 정지 등의 움직임으로 인해

회로 보드에 충격이 가해질 수 있다. 이에 따라 회로 구성 요소의 불안정한 동작, 접점의 이탈 또는 손상 등이 발생할 수 있다.

- 2) 이를 방지하기 위해 충격에 강한 재료로 보호된 회로 보드를 사용하고, 충격 및 진동 감소를 위한 기계적인 보호 장치를 준비하는 게 좋다.

3. 정전기 방지

(1) 내용

정전기는 전자 회로에 해로울 수 있는 주요한 원인 중 하나이다. 정전기 방지를 위해 정전기 방지 소자나 접지 처리를 신중히 고려해야 한다. 특히 로봇과 같이 정전기 발생 가능성이 높은 환경에서는 추가적인 보호 대책이 필요하다.

(2) 사례

(가) 현상

정전기 방지 대책이 없는 회로 보드에 정전기 방전이 발생하면, 정전기 방전에 의한 고전압이 회로 구성 요소에 영향을 줄 수 있고 이에 따라 부품 손상, 회로 동작의 불안정성 또는 데이터 손실 등의 문제가 발생할 수 있다.

(나) 대책

정전기 방지를 위해 회로 보드에는 정전기 방전 보호 회로, 접지 장치, 정전기 방지 소자 등을 포함하는 게 좋다.

4. EMI/RFI 차폐

(1) 내용

EMI(electromagnetic interference)와 RFI(radio frequency interference)는 신호 간섭을 초래할 수 있다. 회로에 충분한 차폐와 필터링을 적용하여 외부 잡음의 영향을 최소화해야 한다. 이를 위해 EMI 필터, 그라운드 플레인 디자인, 적절한 신호 라우팅을 고려해야 한다.

(2) 사례

(가) 사례 1

- 1) 현상: 고주파 신호나 전자기장으로 인해 회로 보드에 EMI (전자기적 유입) 또는 RFI (전파적 유입)가 발생할 경우, 주변 장치에 무선 장애를 일으킬 수 있다.
- 2) 대책: 이를 방지하기 위해 회로 보드에는 EMI/RFI 차폐 코팅, 그라운드 플레인, 필터링 회로, 차폐 케이스 등의 차폐 장치를 사용해야 한다.

(나) 사례 2

- 1) 현상: 민감한 신호 처리 회로가 있는 회로 보드가 주변에 있는 높은 주파수 장비로부터 유입되는 EMI에 노출될 경우, 신호 왜곡이 발생하거나 불필요한 잡음이 섞일 수 있다.

- 2) 대책: 이를 방지하기 위해 회로 보드의 레이아웃 설계, 차폐 장치 및 적절한 그라운드 플레인을 사용하여 EMI/RFI의 영향을 최소화해야 한다.

5. 신호 강도와 잡음 마진

(1) 내용

신호 강도와 잡음 마진은 신호의 신뢰성과 안정성을 보장하기 위해 고려되어야 한다. 충분한 신호 강도와 잡음 마진을 유지하면서 신호의 정확성과 안정성을 향상시킬 수 있다.

(2) 사례

(가) 사례 1

- 1) 현상: 신호의 강도와 잡음 마진을 고려하지 않은 경우, 신호의 약화나 잡음의 증폭으로 인해 신호의 왜곡이 발생할 수 있고 이에 따라 데이터 손실, 오작동 또는 신호 간섭 등의 문제가 발생한다.
- 2) 대책: 적절한 신호 강도와 잡음 마진을 고려하여 회로 보드의 설계와 신호 처리 방식을 설정해야 한다.

(나) 사례 2

- 1) 현상: 신호 전송 경로가 긴 회로 보드에서 고속 데이터 신호가 송수신되는 경우, 신호의 감쇠나 잡음의 증폭으로 인해 신호의 품질이 저하될 수 있다.
- 2) 대책: 이를 방지하기 위해 적절한 신호 강도 보상 기법, 잡음 억제 회로, 신호 재생 회로 등을 사용하여 신호 강도와 잡음 마진을 유지해야 한다.

6. 오작동 방지

(1) 내용

회로가 예기치 않은 오작동을 일으키지 않도록 안정성을 고려해야 한다. 신호 경로의 격리, 회로 보호 기능, 오작동 검출 및 복구 기능 등을 포함하여 신뢰성을 높이는 방법을 고려해야 한다.

(2) 사례

(가) 사례 1

- 1) 현상: 회로 보드에 오작동을 일으킬 수 있는 부적절한 접점, 소자 결함, 신호 간섭 등의 문제가 있는 경우, 회로 보드의 동작이 비정상적으로 동작하거나 전원 공급이 중단될 수 있다.
- 2) 대책: 이를 방지하기 위해 회로 보드에는 신뢰성 높은 부품 및 접점, 테스트 및 검증 과정을 통한 결함 탐지, 신호 간섭 회피를 위한 설계 등의 방법을 적용해야 한다.

(나) 사례 2

- 1) 현상: 통신 회로에서 신호의 손실이나 왜곡으로 인해 데이터 전송 오류가 발생하는

경우, 잘못된 데이터 처리, 통신 연결의 중단 또는 데이터 손실 등의 문제가 발생한다.

- 2) 대책: 이를 방지하기 위해 신호 강도 감지 및 오류 검출 기술, 신뢰성 있는 데이터 전송을 위한 오류 정정 알고리즘 등을 적용해야 한다.

교수 방법

- 제어공학, 마이크로컨트롤러, 회로이론 등 기초적인 내용에 관해 잘 알고 있는지 확인하고 로봇 주제어 장치 하드웨어의 회로 설계에 관해 설명한다.
- 회로 구성 요소의 구조와 상호 관계 등에 관한 시각적 자료를 만들어 이해하기 쉽게 설명한다.
- 신호 간섭과 노이즈에 관한 시각적 자료를 만들어 이해하기 쉽게 설명한다.
- 데이터 통신 회로 도면을 실제로 제작하고 관리하는 실습을 하도록 지도한다.
- 부품 간 클럭 오류나 노이즈 발생 원인에 관해 다양한 사례 수집을 유도한다.

학습 방법

- 수강의 준비를 위해 제어공학, 마이크로컨트롤러, 회로이론 등 회로 설계의 기초적인 내용에 관해 사전에 충분히 학습한다.
- 학습에 임하기 전에 회로 구성 요소의 구조와 상호 관계 등에 관한 기초 내용을 학습한다.
- 학습 전에 신호 간섭과 노이즈에 관해 미리 검색하고 정리한다.
- 데이터 통신 회로 도면을 실제로 제작하고 관리하는 실습에 충실히 참여하고 그 내용을 정리한다.
- 부품 간 클럭 오류나 노이즈 발생 원인과 해결책, 예방책에 관해 다양한 사례를 수집하고 연구한다.

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 회로 설계	- 선정된 MCU 처리부와 주변기기의 사양을 확인하고 부품을 배치할 수 있다.			
	- 결정된 메모리용량, 신호처리, 데이터 타이밍 인터페이스를 확인할 수 있다.			
	- 부품의 배치에 따라 신호의 간섭과 노이즈를 고려하여 MCU부 회로를 설계할 수 있다.			
	- MCU 회로설계에 관련된 설계도면을 관리할 수 있다.			

평가 방법

- 서술형 시험

학습 내용	평가 항목	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 회로 설계	- 주제어 장치의 다양한 부품의 사양을 확인하고 배치할 수 있는 능력			
	- 다양한 인터페이스를 설계하고 확인할 수 있는 능력			
	- 다양한 클럭 오류나 노이즈 등에 관한 원인 분석, 해결책, 예방책의 숙달 여부			

• 평가자 질문

학습 내용	평가 항목	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 회로 설계	- 주제어 장치의 다양한 부품의 사양을 확인하고 배치한 경험 과 지식 보유 여부			
	- 다양한 인터페이스를 설계하고 오류를 식별할 수 있는 능력			
	- 다양한 클럭 오류나 노이즈 등에 관한 원인 분석, 해결책, 예방책의 숙달 여부			

• 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 회로 설계	- 주제어 장치의 다양한 부품의 사양을 확인하고 배치할 수 있는 능력			
	- 다양한 인터페이스를 설계하고 제작한 경험의 보유 여부			
	- 다양한 클럭 오류나 노이즈 등에 관한 원인 분석, 해결책, 예방책의 경험 보유 여부			

피드백

1. 서술형 시험

- 회로 설계에서 필수적인 부분과 선택적인 부분을 구분하여 서술형 시험 결과를 평가하고 그 결과를 확인할 수 있도록 한다.
- 평가 결과 성적이 저조한 학생은 주제어 장치회로 설계에 관한 보고서 제출 기회를 부여한다.
- 고득점 학습자에게는 최신 설계 동향에 관해 안내하고 보다 선진화된 기법을 인식할 수 있게 한다.

2. 평가자 질문

- 회로 설계에 관한 질문 목록을 만들어 학습 시간에 피평가자에게 질문을 한 후 답변에 관하여 조언한다.
- 답변을 잘하지 못한 학생은 주제어 장치 회로 설계에 관한 보고서 제출 기회를 부여한다.
- 고득점 학습자에게는 SI에 관한 개념과 사례를 소개한다.

3. 평가자 체크리스트

- 회로를 설계하기 위하여 수집된 자료 품질에 관한 수준을 서로 비교하고 평가한 결과를 공유한다.
- 체크리스트 결과의 적정성, 과정 등을 평가하고 미비점을 지적한 후 추가 학습 결과를 제출하도록 한다.
- 고득점 학습자에게는 최신 설계 기법을 소개한다.

2-1. 로봇 주제어 장치(MCU) 하드웨어 검증

학습 목표

- 개념설계 단계에서 결정된 기능을 검증하기 위하여 항목, 순서, 방법을 결정할 수 있다.
- 전원과 클럭을 인가하여 MCU가 정상적으로 작동하는지 판단할 수 있다.
- 개념설계 단계에서 결정된 기능을 검증하기 위하여 MCU 하드웨어 검증코드를 작성할 수 있다.
- 검증작업을 수행하고 문제점을 분석하여 검증 결과 보고서를 작성할 수 있다.

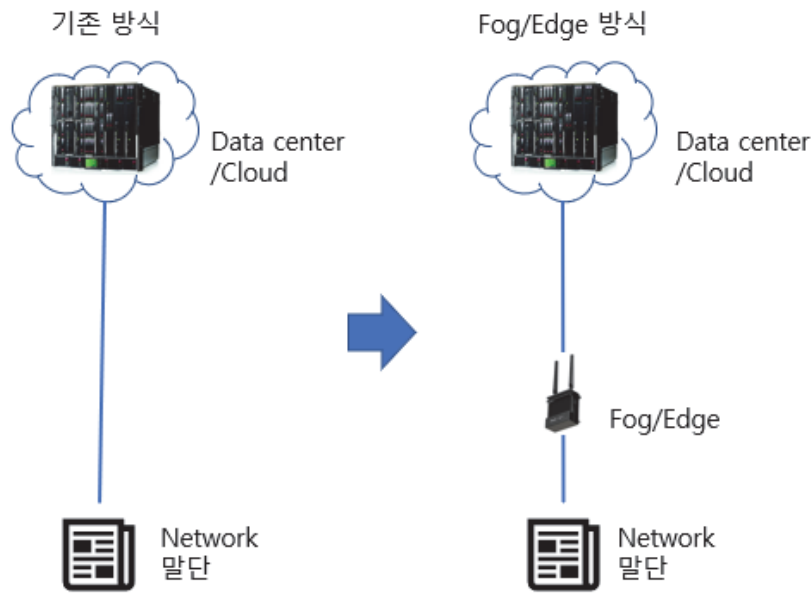
필요 지식 /

① 포그 컴퓨팅

1. 개념과 배경

포그 컴퓨팅이란 네트워크를, 데이터를 주고받는 사용자와 물리적으로 가까운 장소에 분산 배치한다는 개념이다. 데이터 처리를 클라우드에만 의존하는 것이 아니라, 아래의 [그림 2-1]과 같이 데이터가 생성되는 곳과 물리적으로 가까운 곳에서 애플리케이션을 내장한 별도의 장비를 배치해서 더 많은 데이터를 효율적으로 활용할 수 있도록 하는 것이다. 이 포그 컴퓨팅이 도입되는 배경으로는 다음과 같은 필요성이 있었다.

- 낮은 지연 시간 응답성 필요
- 복원력
- 보안
- 대역폭보다 빠르게 증가하는 데이터



출처: 집필진 제작(2023)
[그림 2-1] 포그 컴퓨팅

2. 포그 컴퓨팅의 발생

클라우드가 보편화되면 IoT가 많이 그리고 바로 구현될 것으로 생각하지만, 현실적으로는 스마트 기기와 IoT 기기가 급속히 증가함에 따라 클라우드를 통한 업무 처리가 어려워졌다. 이에 관해 클라우드에 업무가 몰리는 것을 단속 또는 절감하고 사물인터넷이 생산하는 대량의 데이터를 사물의 네트워크 말단에서, 혹은 그 사물과 인터넷 사이에 위치한 기기에서 저장하고 처리하는 방안을 강구하게 되었다. 이것을 시*코(C***)에서는 포그 컴퓨팅이라 명명했다. 클라우드(구름)가 하늘 위 먼 곳에 있고 추상적인 느낌이 있지만, 포그(안개)는 지면과 가깝고 실제 생활이 있는 바로 옆에 같이 있다는 개념이다.

3. 포그 컴퓨팅의 특징

포그 컴퓨팅에서는 클라우드 방식과 다르게 중요한 사항만 클라우드나 중앙 서버에 처리를 의뢰하고 대부분의 사소한 업무는 네트워크 말단에서 자율적으로 연산하고 처리하여 실시간으로 대응한다. 중앙에서 모든 정보와 기능을 독점하고 통제하는 클라우드 컴퓨팅 방식에 비해 말단 기기들이 독립성을 가진다는 것이 차이점이다.

4. 포그 컴퓨팅의 활용

응급 목적의 자동차에게 목적지까지 막힘없이 이동할 수 있도록 신호등에 조치를 하는 스마트 교통 신호를 만들거나, 철로의 마모나 균열 또는 도로의 포트홀이나 균열 등을 실시간으로 관찰하고 기록하며 동시에 중앙 데이터 센터로 무선 송신하여 저장하고 처리하여 교체 또는 시공 등의 대책 수립에 기반을 제공하기도 한다. 송유관의 불량 상태나 가스 누출을 검지해 사고를 미리 예방하는 등 사회 기반 시설의 안전성을 높이는 효과도 기대할 수 있다.

‘포그 컴퓨팅(fog computing)’과 ‘엣지 컴퓨팅(edge computing)’

포그 컴퓨팅은 크고 튼튼한 서버들로 이루어진 것이 아니라 더 작고 분산된 컴퓨터들로 구성되어 있으며, CCTV, 가전제품, 자판기, 자동차 등을 비롯하여 우리의 생활 곳곳에 산재되어 있는 바로 그 컴퓨터들로 구성된다. 여기서 컴퓨터라 하면 바로 떠오르는 일반 가정용 또는 사무용 컴퓨터가 아니라 보다 훨씬 소형의 산업용 컴퓨터를 말한다.

시*코(C***)만 포그 컴퓨팅을 제창하는 것이 아니다. 다른 제조사들도 같은 개념을 현실화시켰으며, 아**엠(I*M)의 엣지 컴퓨팅(edge computing)이 그 중 대표적인 개념이다.

여기서의 ‘엣지’는 말 그대로 네트워크의 끝단 즉, 인터넷 세계가 끝나고 현실 세계가 시작되는 부분을 말하는데, 데이터 센터나 클라우드 서버가 네트워크의 중심에 있다 할 때 스마트폰, 감시카메라, 가정용 컴퓨터 등이 네트워크 끝단에 위치한 기기들이이며 엣지 컴퓨팅을 위한 장비로 라****이로 대별되는 미니 PC, SBC(Single Board Computer), 엣지 컴퓨터 등의 장비들이 이미 출시되어 있다.

출처 : 민세아(2014. 10. 8.), “클라우드에서 내려와 지상으로! 포그 컴퓨팅”, 보안뉴스.

수행 내용 1 / MCU 보드의 방열판 교체하기

재료·자료

- Single Board Computer 보드(TINKER BOARD R2.0)
- Open Case DIY Kit(FAN, 방열판)
- Tinker Board 매뉴얼

기기(장비 · 공구)

- 컴퓨터
- 시스템 온도 측정 솔루션
- 전자 장비 설치 공구 세트(드라이버 등)

안전 · 유의 사항

- 모든 공정이 끝나고 안전이 확인되기 전까지 전력을 인가하지 않는다.

- 보드를 손으로 만질 때 도전 부분에 닿지 않도록 주의한다.
- 부품을 부착할 때 부착 방법을 적절히 선택하고 무리하게 힘을 주지 않도록 한다.
- 신중하게 작업하여 작업 도중 주변의 부품을 건드리지 않도록 한다.
- 교체 후 온도 측정 등 결과 확인 시에, 측정 시간 동안 측정 조건이 유지되도록 주변과 상황을 관리한다.

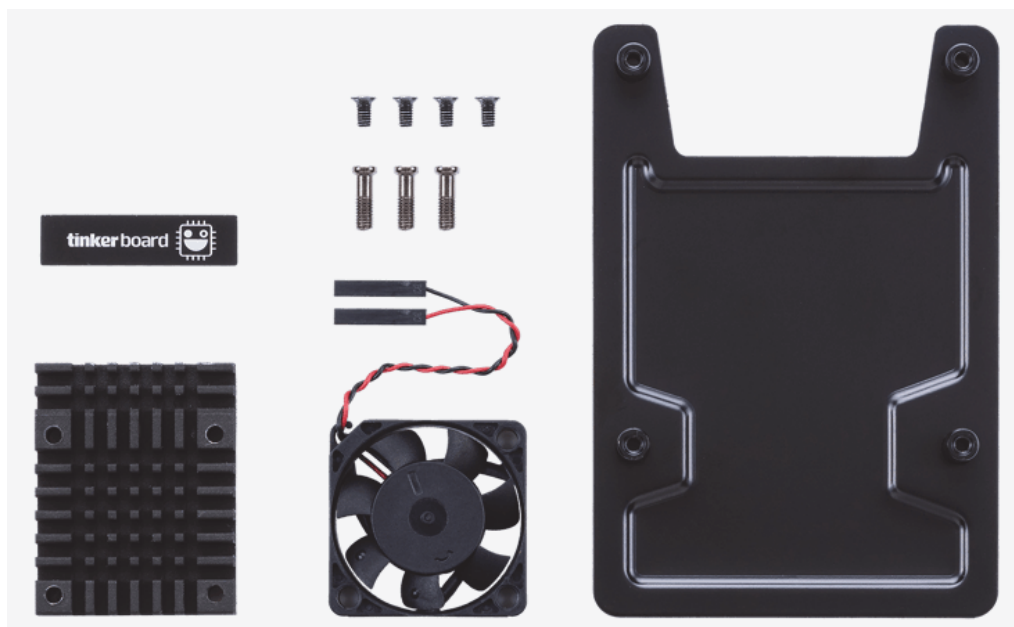
수행 순서

① MCU 하드웨어의 방열판과 팬을 교체하는 방법을 알아본다.

작동 중에 재부팅되거나 화면이 멈추는 등의 현상이 발생하기에 다양한 조사와 검증을 시도하던 중 방열판이 지나치게 뜨거운 걸 확인했다. 제조사에 무상 교체를 신청하지 않고 대신 기술 문의를 하여 기술 지원을 통해 회신받은 바대로 수행했다.

1. 대체 방열판과 팬 수배

마더보드 제조사 홈페이지의 Open Case DIY Kit을 구매한다. 수급이 불안정하거나 할 경우, 라즈베리파이용 FAN을 사용할 수 있다.



출처: ASUS IoT 홈페이지. "ASUS TINKER BOARD SERIES". <https://tinker-board.asus.com/accessories/tinker-kit.html>에서 2023. 6. 2. 검색)

[그림 2-2] open case DIY kit

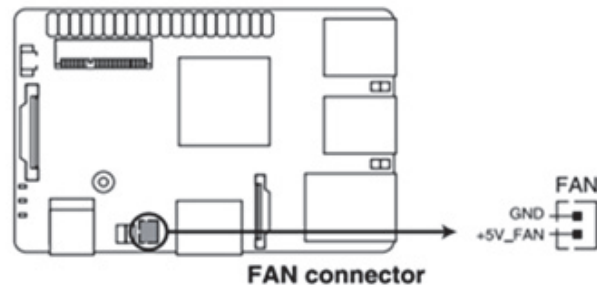
2. 부품 교체

(1) 작업

아래 [그림 2-3]을 참고하여 일반 마더보드의 FAN 구성과 동일하게 FAN의 2-Pin Connector를 Pin-map에 1:1 mapping하여 접지한다. DC Type 이므로 따로 PWM 제어는 불가하다.

6. DC Fan header

The DC Fan header allows you to connect a fan to actively cool the system.



Connector Type	JST PH 2P 2.00mm
Reference PN:	JST, PHR-2

출처: 집필진 제작(2023)

[그림 2-3] fan header의 pin 맵

(2) 주의사항

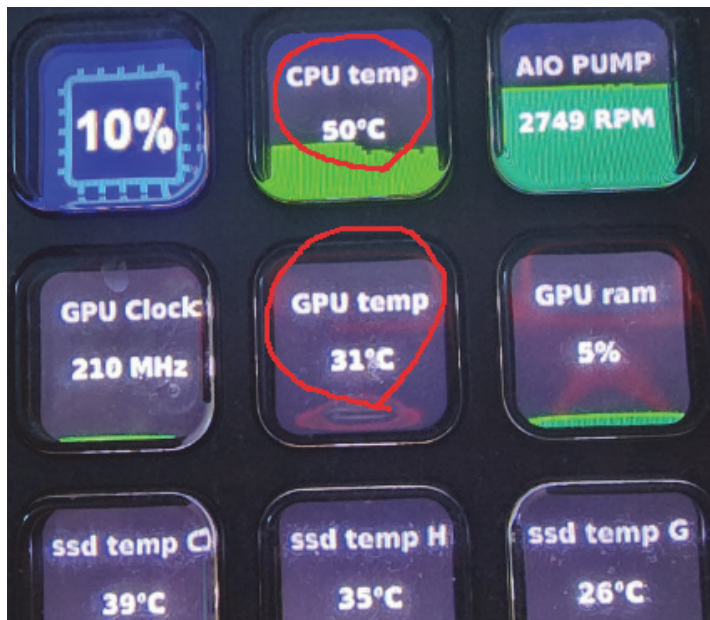
상기 항목들은 점퍼가 아니므로 커넥터에 점퍼 캡을 씌우면 안 된다. 연결되는 케이블은 커넥터에 충분히 삽입되도록 한다.

추가로 FAN Header 뿐만 아니라, GPIO (VCC5V, GND)에 접지하여 FAN 구성도 가능하다.

Pin definition	40P GPIO	Pin definition
VCC3.3_IO	1	VCC5V
GPIO2_B1/I2C6_SDA	3	GND
GPIO2_B2/I2C6_SCL	5	GPIO2_C1/UART0_TX
GPIO0_B0/CLKOUT	7	GPIO2_C0/UART0_RX
GND	9	GPIO3_D1/I2S0_SCLK
GPIO2_C3/UART0_RTSN	11	GND
GPIO2_C5/SPI5_TXD	13	GPIO2_C6/SPI5_CLK
GPIO2_C4/SPI5_RXD	15	GPIO2_C7/SPI5_CSN0
VCC3.3_IO	17	GND
GPIO1_B0/SPI1_TXD/UART4_TX	19	GPIO3_D4/I2S0_SDI1SDO3
GPIO1_A7/SPI1_RXD/UART4_RX	21	GPIO1_B2/SPI1_CSN0
GPIO1_B1/SPI1_CLK	23	GPIO0_A6/PWM3A_IR
GND	25	GPIO2_B0/I2C7_SCL
GPIO2_A7/I2C7_SDA	27	GND
GPIO3_D6/I2S0_SDI1SDO1	29	GPIO4_C2/PWM0
GPIO3_D5/I2S0_SDI1SDO2	31	GND
GPIO4_C6/PWM1	33	GPIO2_C2/UART0_CTSN
GPIO3_D1/I2S0_LRCK	35	GPIO3_D3/I2S0_SDI0
GPIO4_C5/SPDIF_TX	37	GPIO3_D7/I2S0_SDO0
GND	39	
	40	

출처: 집필진 제작(2023)
[그림 2-4] 40P GPIO 구성

3. 교체 결과 확인



출처: 집필진 제작(2023)
[그림 2-5] 시스템 온도 확인

부품 대체의 결과를 확인하기 위해 교체 전 기존 방열판 사용 시의 온도를 측정하고 대체 후에 다시 확인하였다. 설치 직후에는 정상 온도를 보여줬다. 그래서 장시간 다양한 작업을 자동으로 돌려 보고 종합적으로 결과를 정리하였다.

〈표 2-1〉 방열판과 팬 교체 후 온도 측정

	기존 방열판	새 방열판
내재된 기본 작업만 수행할 때	65도 내외	60도 이하
장시간 고도의 부하를 걸었을 때	75도 이상	69도 내외

새 방열판으로 인한 온도 차가 5~6도 정도임을 확인하였다. 그리고 최고 온도를 기록한 후 다시 정상 온도로 내려오는 데 걸리는 시간이 평균 60분 내외인 것도 확인하였다.

수행 내용 2 / MCU의 주요 기능별 검증 항목에 따라 검증 코드 짜기

재료·자료

- 소프트웨어 툴
- 설계대로 제작된 MCU (Raspberry Pi Pico 기반)
- MCU와 컴퓨터 사이의 통신 솔루션

기기(장비 · 공구)

- 컴퓨터

안전 · 유의 사항

- 요구 사항 명세를 확실히 이해하고 명세와 작업 목적이 일치하는지 확인한다.
- 검증의 목적을 완전히 이해하여 불필요한 작업을 배제한다.
- 예외 상황을 모의하거나, 잘못된 입력을 시뮬레이션하여 올바른 에러 처리를 검증해야 한다.
- 다양한 입력값과 범위, 경계 조건, 오류 케이스 등을 고려하여 적절한 테스트 데이터를 만들고 사용해야 한다.

- 검증 코드를 작성할 때 가독성과 유지 보수성을 고려해야 한다. 코드를 쉽게 이해할 수 있도록 주석을 추가하고, 변수와 함수의 이름을 명확하게 지정해야 한다. 또한, 검증 코드를 수정하거나 추가할 때 다른 개발자가 쉽게 이해하고 수정할 수 있도록 작성해야 한다.

수행 순서

① MCU의 주요 기능을 검증하기 위한 검증 항목을 알아본다.

1. Raspberry Pi Pico MCU의 검증을 위한 검증 항목과 검증 방법

Raspberry Pi Pico MCU의 주요 기능을 검증하기 위한 검증 항목과 해당 검증 방법은 다음과 같다. 이는 일반적인 검증 항목으로, 특정 요구사항이나 사용 사례에 따라 추가적인 검증이 필요할 수 있다. 또한, 순서는 상황에 따라 유연하게 조정될 수 있다.

(1) 전원 공급 검증

- 연결된 전원 공급 장치로부터 Pico에 전원이 제대로 공급되는지 확인
- LED 등을 사용하여 전원 상태를 시각적으로 확인

(2) GPIO (일반 입력 및 출력) 검증

- 각 GPIO 핀을 사용하여 입력 및 출력 동작을 수행하는 테스트 코드 작성
- GPIO 핀 상태를 모니터링하고, 올바른 동작을 확인

(3) 아날로그 입력 검증

- 아날로그 입력용 핀을 사용하여 아날로그 센서값을 읽어오는 테스트 코드 작성
- 아날로그 센서값을 측정하고, 올바른 값을 확인

(4) PWM 출력 검증

- PWM 출력 핀을 사용하여 LED 밝기 조절 등 PWM 신호를 출력하는 테스트 코드 작성
- PWM 신호의 주기와 듀티 사이클을 변경하고, 예상한 동작이 이루어지는지 확인

(5) UART 통신 검증

- UART 통신 핀을 사용하여 시리얼 데이터 송수신을 수행하는 테스트 코드 작성
- 시리얼 모니터 등 외부 장치를 사용하여 데이터 송수신을 확인

(6) I2C 통신 검증

- I2C 통신 핀을 사용하여 외부 디바이스와의 통신을 수행하는 테스트 코드 작성
- I2C 디바이스와 데이터 송수신을 확인하고, 올바른 데이터를 주고받는지 확인

(7) SPI 통신 검증

- SPI 통신 핀을 사용하여 외부 디바이스와의 통신을 수행하는 테스트 코드 작성
- SPI 디바이스와 데이터 송수신을 확인하고, 올바른 데이터를 주고받는지 확인
- 외부 인터럽트 검증:

(8) 외부 인터럽트를 트리거하는 핀과 인터럽트 핸들러를 사용하는 테스트 코드 작성

외부 인터럽트가 발생할 때 적절한 동작이 수행되는지 확인

(9) 타이머 및 카운터 검증

- 타이머 또는 카운터 기능을 사용하여 시간 측정 또는 주기적인 작업을 수행하는 테스트 코드 작성
- 정확한 시간 간격이 유지되는지 또는 예상한 작업이 주기적으로 수행되는지 확인

(10) 외부 메모리 인터페이스 검증

외부 메모리 장치 (예: SPI Flash)와의 데이터 송수신을 테스트하는 코드 작성
데이터를 외부 메모리에 저장하고, 읽어오는 동작을 확인

② 예제로 기능별 검증 코드를 구성해 본다.

1. Python 기반

각각의 검증 항목에 관한 Python 코드 예제는 다음과 같다. 필요에 따라 코드를 수정하고, 특정 기능에 맞게 추가 검증을 수행할 수 있다. 각 코드 예제는 해당 기능을 검증하고 예상된 동작을 확인할 수 있도록 작성되었다.

(1) 전원 공급 검증

```
python
Copy code
import machine
led = machine.Pin(25, machine.Pin.OUT)
led.value(1) # LED 켜기
```

(2) GPIO (일반 입력 및 출력) 검증

```
python
Copy code
import machine
import time
gpio_pin = machine.Pin(16, machine.Pin.OUT)
gpio_pin.on() # GPIO 핀 상태를 HIGH로 설정
```

```
time.sleep(1)
gpio_pin.off() # GPIO 핀 상태를 LOW로 설정
```

(3) 아날로그 입력 검증

```
python
Copy code
import machine
adc = machine.ADC(26) # 아날로그 입력용 핀 번호
analog_value = adc.read()
print('Analog value:', analog_value)
```

(4) PWM 출력 검증

```
python
Copy code
import machine
import time
pwm_pin = machine.Pin(14)
pwm = machine.PWM(pwm_pin)
pwm.freq(1000) # PWM 주파수 설정
pwm.duty_u16(32768) # 듀티 사이클 설정 (0~65535)
time.sleep(2)
pwm.deinit() # PWM 정지
```

(5) UART 통신 검증

```
python
Copy code
import machine
uart = machine.UART(0, baudrate=9600, tx=0, rx=1)
uart.write('Hello, UART!')
response = uart.read()
print('Response:', response)
```

(6) I2C 통신 검증

```
python
Copy code
import machine
import time
i2c = machine.I2C(0)
```

```

i2c.start()
i2c.writeto(0x48, b'\x00') # I2C 주소 및 데이터 전송
time.sleep(0.1)
data = i2c.readfrom(0x48, 2)
i2c.stop()
print('I2C data:', data)

```

(7) SPI 통신 검증

```

python
Copy code
import machine
import time
spi = machine.SPI(0)
spi.init(baudrate=1000000, polarity=0, phase=0)
spi.write(b'Hello, SPI!')
response = spi.read(10)
print('Response:', response)

```

(8) 외부 인터럽트 검증

```

python
Copy code
import machine
import time
def handle_interrupt(pin):
    print('External interrupt occurred!')
ext_pin = machine.Pin(4, machine.Pin.IN, machine.Pin.PULL_UP)
ext_pin.irq(handler=handle_interrupt, trigger=machine.Pin.IRQ_FALLING)
time.sleep(5)
ext_pin.irq(handler=None) # 인터럽트 핸들러 해제

```

(9) 타이머 및 카운터 검증

```

python
Copy code
import machine
import utime
start_time = utime.ticks_ms()
while utime.ticks_diff(utime.ticks_ms(), start_time) < 5000:

```

```
# 5초 동안 작업 수행
```

```
pass
```

(10) 외부 메모리 인터페이스 검증:

```
python
```

```
Copy code
```

```
import machine
```

```
import time
```

```
spi = machine.SPI(0)
```

```
spi.init(baudrate=1000000, polarity=0, phase=0)
```

```
spi.write(b'\x03\x00\x00\x00\x00') # 외부 메모리로부터 데이터 읽기
```

```
time.sleep(0.1)
```

```
response = spi.read(10)
```

```
print('Response:', response)
```

2. C/C++ 기반

각각의 검증 항목에 관한 C/C++ 코드 예제는 다음과 같다. 필요에 따라 코드를 수정하고, 특정 기능에 맞게 추가 검증을 수행할 수 있다. 각 코드 예제는 해당 기능을 검증하고 예상된 동작을 확인할 수 있도록 작성되었다.

(1) 전원 공급 검증

```
cpp
```

```
Copy code
```

```
#include "pico/stdlib.h"
```

```
int main() {
```

```
    gpio_init(25);
```

```
    gpio_set_dir(25, GPIO_OUT);
```

```
    gpio_put(25, 1); // LED 켜기
```

```
    return 0;
```

```
}
```

(2) GPIO (일반 입력 및 출력) 검증

```
cpp
```

```
Copy code
```

```
#include "pico/stdlib.h"
```

```
#include <unistd.h>
```

```
int main() {
```

```
    gpio_init(16);
```

```

    gpio_set_dir(16, GPIO_OUT);
    gpio_put(16, 1); // GPIO 핀 상태를 HIGH로 설정
    sleep(1);
    gpio_put(16, 0); // GPIO 핀 상태를 LOW로 설정
    return 0;
}

```

(3) 아날로그 입력 검증

```

cpp
Copy code
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include <stdio.h>
int main() {
    adc_init();
    adc_gpio_init(26); // 아날로그 입력용 핀 번호
    adc_select_input(0); // 아날로그 입력 선택

    uint16_t analog_value = adc_read();
    printf("Analog value: %d\n", analog_value);
    return 0;
}

```

(4) PWM 출력 검증

```

cpp
Copy code
#include "pico/stdlib.h"
#include <stdio.h>
#include <unistd.h>
int main() {
    gpio_init(14);
    gpio_set_function(14, GPIO_FUNC_PWM);
    uint slice_num = pwm_gpio_to_slice_num(14);
    pwm_set_wrap(slice_num, 65535);
    pwm_set_chan_level(slice_num, PWM_CHAN_A, 32768);
    pwm_set_enabled(slice_num, true);
}

```



```

    sleep(2);
    pwm_set_enabled(slice_num, false);
    pwm_set_chan_level(slice_num, PWM_CHAN_A, 0);
    return 0;
}

```

(5) UART 통신 검증

```

cpp
Copy code
#include "pico/stdlib.h"
#include "hardware/uart.h"
#include <stdio.h>
#define UART_ID uart0
#define BAUD_RATE

```

(6) I2C 통신 검증

```

cpp
Copy code
#include "pico/stdlib.h"
#include "hardware/i2c.h"
#include <stdio.h>
#include <unistd.h>
int main() {
    i2c_init(i2c0, 100000);
    gpio_set_function(2, GPIO_FUNC_I2C);
    gpio_set_function(3, GPIO_FUNC_I2C);
    gpio_pull_up(2);
    gpio_pull_up(3);
    i2c_write_blocking(i2c0, 0x48, NULL, 0, true); // I2C 주소 전송
    sleep_ms(100);
    uint8_t data[2];
    i2c_read_blocking(i2c0, 0x48, data, 2, false); // 데이터 수신
    printf("I2C data: %d, %d\n", data[0], data[1]);
    return 0;
}

```

(7) SPI 통신 검증

cpp

Copy code

```
#include "pico/stdlib.h"
#include "hardware/spi.h"
#include <stdio.h>
#include <unistd.h>

int main() {
    spi_init(spi0, 1000000);
    gpio_set_function(0, GPIO_FUNC_SPI);
    gpio_set_function(1, GPIO_FUNC_SPI);
    gpio_set_function(2, GPIO_FUNC_SPI);
    gpio_set_function(3, GPIO_FUNC_SPI);

    spi_write_blocking(spi0, "Hello, SPI!", 12);
    uint8_t response[10];
    spi_read_blocking(spi0, response, 10);
    printf("Response: %s\n", response);
    return 0;
}
```

(8) 외부 인터럽트 검증

cpp

Copy code

```
#include "pico/stdlib.h"
#include "hardware/gpio.h"
#include <stdio.h>
#include <unistd.h>

void handle_interrupt() {
    printf("External interrupt occurred!\n");
}

int main() {
    gpio_init(4);
    gpio_set_dir(4, GPIO_IN);
    gpio_pull_up(4);
```

```

        gpio_set_irq_enabled_with_callback(4,      GPIO_IRQ_EDGE_FALL,      true,
&handle_interrupt);
        sleep(5);
        gpio_set_irq_enabled(4, GPIO_IRQ_EDGE_FALL, false);
        return 0;
    }

```

(9) 타이머 및 카운터 검증

```

cpp
Copy code
#include "pico/stdlib.h"
#include "pico/time.h"
#include <stdio.h>
int main() {
    absolute_time_t start_time = get_absolute_time();
    while (absolute_time_diff_us(get_absolute_time(), start_time) < 5000000) {
        // 5초 동안 작업 수행
    }
    return 0;
}

```

(10) 외부 메모리 인터페이스 검증

```

cpp
Copy code
#include "pico/stdlib.h"
#include "hardware/spi.h"
#include <stdio.h>
#include <unistd.h>
int main() {
    spi_init(spi0, 1000000);
    gpio_set_function(0, GPIO_FUNC_SPI);
    gpio_set_function(1, GPIO_FUNC_SPI);
    gpio_set_function(2, GPIO_FUNC_SPI);
    gpio_set_function(3, GPIO_FUNC_SPI);
    spi_write_blocking(spi0, "\x03\x00\x00\x00\x00", 5); // 외부 메모리로부터
데이터 읽기

```

```
    sleep_ms(100);  
    uint8_t response[10];  
    spi_read_blocking(spi0, response, 10);  
    printf("Response: %s\n", response);  
    return 0;  
}
```

Python 코드와 C/C++ 코드의 비교

1. 비교 특징

Python과 C/C++은 각각 다른 특징과 장단점을 가지고 있다. 이에 따라 어떤 언어가 효율적인지는 다음과 같은 요소에 따라 달라진다. 다만, 이는 일반 개발자들의 일부의 의견일 수 있으므로 상황에 따라 달라질 수 있다.

(1) 성능

C/C++은 일반적으로 Python보다 더 빠른 실행 속도를 가진다. 특히 하드웨어와 밀접하게 상호 작용해야 하는 작업이나 대용량 데이터 처리와 같은 성능이 중요한 작업에는 C/C++이 더 적합하다.

(2) 개발 속도

일반적으로 Python은 C/C++보다 문법이 간단하고 읽기 쉬운 언어로, 개발 속도가 빠를 수 있다. 특히 프로토타이핑이나 간단한 작업에는 Python이 더 편리할 수 있다.

(3) 하드웨어 접근

C/C++은 하드웨어 접근에 더 적합한 언어로, 하드웨어 레지스터에 직접 접근하거나 특정 하드웨어 기능을 활용하는 작업에는 C/C++이 더 효과적이다.

2. 선택 시 고려 사항

특징에 따라 검증 과정에서 어떤 언어를 선택할지는 다음을 고려해야 한다.

(1) 작업의 퍼포먼스 요구 사항

실시간 성능이 중요하거나 대량의 데이터 처리가 필요한 경우 C/C++을 고려할 수 있다.

(2) 개발 생산성

신속한 프로토타이핑이 필요하거나 간단한 스크립트 작성이 필요한 경우 Python이 유리할 수 있다.

(3) 기능 및 라이브러리 지원

특정 기능이나 라이브러리가 필요한 경우 해당 언어에서 지원하는지 확인해야 한다.

(4) 개발자의 선호도 및 경험

개발자가 특정 언어에 익숙하거나 선호하는 언어가 있다면 해당 언어를 선택하는 것이 좋다.

3. 비교 결론

(1) 검증 코드의 특성과 목적

전술한 선택 시 고려 사항에 따라서, 검증 코드의 특성과 목적에 따라 언어 선택이 달라질 수 있다.

(가) C/C++

높은 성능이 필요하거나 하드웨어 접근이 많이 필요한 경우 C/C++을 사용하는 것이 좋다.

(나) Python

개발 속도나 간편한 문법을 우선시할 때는 Python을 선택하는 것이 적합할 수 있다.

(2) 효율성

마찬가지로 효율성 역시 상황에 따라 다를 수 있다.

(가) C/C++

C/C++은 Python에 비해 실행 속도가 빠르며, 시스템 리소스를 더욱 효율적으로 활용할 수 있다. 따라서 퍼포먼스에 중점을 두는 작업이나 실시간 성능이 필요한 작업에는 C/C++이 더 적합할 수 있다.

(나) Python

Python은 C/C++보다 개발 속도가 빠르고 문법이 간결하여 개발 생산성을 높일 수 있다. Python은 동적 타이핑을 지원하고, 풍부한 라이브러리와 프레임워크가 있어서 개발이 편리하다. 또한, 프로토타이핑이나 간단한 스크립트 작성에는 Python이 유리하다.

종합해서 보면, 성능에 초점을 두거나 시스템 리소스를 효율적으로 활용해야 할 때는 C/C++을 선택하는 것이 좋을 수 있지만, 개발 생산성이나 간단한 작업에는 Python이 더 적합할 수 있다.

③ 동작 측면에서 부분합 적인 검증을 시도한다.

1. 서보 모터의 동작

아래 코드는 C/C++ 프로그래밍 언어로 작성된 RCSERVO 헤더 파일의 내용이다. 이 코드는 전처리기 지시문인 `#ifndef`, `#define`, `#endif`를 사용하여 헤더 파일의 중복 포함을 방지하기 위한 가드(Guard) 조건문으로 알려져 있다.

(1) 선언과 정의

내용을 살펴보면 다음과 같은 선언과 정의가 포함되어 있다.

(가) `MIN_WIDTH`, `MAX_WIDTH`

정수형 상수로, 서보 모터의 최소 및 최대 폭을 나타낸다.

(나) `MIN_WIDTH2`, `MAX_WIDTH2`

또 다른 정수형 상수로, 다른 서보 모터의 최소 및 최대 폭을 나타낸다.

(다) `MIN_WIDTH3`, `MAX_WIDTH3`

또 다른 정수형 상수로, 또 다른 서보 모터의 최소 및 최대 폭을 나타낸다.

(라) `NEUTRAL_WIDTH`, `NEUTRAL_WIDTH2`, `NEUTRAL_WIDTH3`

서보 모터의 중립 상태(가운데 위치)를 나타내는 값으로, 최소 폭과 최대 폭의 중간 값으로 정의된다.

이 코드는 서보 모터의 폭(파형 신호의 펄스 폭) 범위와 중립 상태에 관한 상수를 정의하고 있다. 이러한 상수들은 서보 모터를 제어하는 데 사용될 수 있으며, 코드의 다른 부분에서 이러한 상수들을 참조하여 서보 모터를 원하는 위치로 제어할 수 있다.

(2) 코드

```
#ifndef __RCSERVO_H__  
#define __RCSERVO_H__
```

```
#define MIN_WIDTH          700  
#define MAX_WIDTH         2700  
#define MIN_WIDTH2        500  
#define MAX_WIDTH2        2500  
#define MIN_WIDTH3        1000  
#define MAX_WIDTH3        2000
```

```
#define NEUTRAL_WIDTH (MAX_WIDTH+MIN_WIDTH)/2  
#define NEUTRAL_WIDTH2 (MAX_WIDTH2+MIN_WIDTH2)/2
```

```
#define NEUTRAL_WIDTH3      (MAX_WIDTH3+MIN_WIDTH3)/2

#endif
```

수행 tip

- 참고로, 위의 코드는 가드(Guard) 조건문으로 헤더 파일의 중복 포함을 방지하기 위해 사용된다. 이를 통해 컴파일러에게 한 번만 포함되도록 보장하고, 중복된 정의나 선언으로 인한 컴파일 오류를 방지할 수 있다.

2. 버튼의 동작 상태 검증

아래 코드 역시 C/C++ 프로그래밍 언어로 작성된 BUTTON 헤더 파일의 내용이다. 이 코드 역시 헤더 파일의 중복 포함을 방지하기 위한 가드(Guard) 조건문으로 알려져 있다.

(1) 선언과 정의

내용을 살펴보면 다음과 같은 선언과 정의가 포함되어 있다:

(가) BTN_SW0, BTN_SW1, BTN_SW2, BTN_SW3

8비트 정수형 상수로, 각각 버튼 SW0, SW1, SW2, SW3에 관한 비트 마스크값을 나타낸다.

(나) NO_BTN

8비트 정수형 상수로, 버튼이 눌리지 않았음을 나타내는 값이다.

(다) BTN_MASK

버튼들에 관한 비트마스크를 포함하는 8비트 정수형 상수이다.

(2) 함수의 선언

또, 다음과 같은 함수 선언이 있다:

(가) BtnInit()

버튼 초기화를 위한 함수로, 버튼을 사용하기 전에 초기화 작업을 수행할 수 있다.

(나) BtnInput()

버튼 입력값을 반환하는 함수로, 버튼이 눌렸는지를 판단하기 위해 사용된다.

(다) Btn_Input_Press()

버튼 입력을 검사하고 눌린 버튼에 관한 정보를 제공하는 함수로, 눌린 버튼의 비트 마스크값을 반환한다.

(라) read_adc()

ADC(아날로그-디지털 변환기)에서 아날로그 입력값을 읽어오는 함수로, 주어진 아날로그 입력에 관한 값을 반환한다.

이 코드는 버튼 입력을 관리하고 버튼 상태를 확인하기 위한 기능을 제공한다. 버튼의 초기화, 입력 상태 확인, 눌린 버튼 검출 등의 작업을 수행할 수 있다. 이를 통해 버튼 입력에 따른 동작을 제어하는 등의 기능을 구현할 수 있다.

④ 종합적인 동작을 검증하는 코드를 작성해 본다.

시도하는 동작은 버튼 및 조이스틱으로부터의 입력에 따라 서보 모터의 각 채널 너비를 조정하고 LCD에 그 정보를 표기하는 것이다.

1. 코드 소개

(1) avr 관련 헤더

아래 해당 코드는 AVR 마이크로컨트롤러를 위한 C/C++ 프로그램으로 주요 AVR 관련 헤더 파일들을 포함하고 있다.

- AVR 입출력 라이브러리인 "avr/io.h",
- 인터럽트 관련 라이브러리인 "avr/interrupt.h",
- 딜레이 함수를 제공하는 라이브러리인 "util/delay.h"
- 사용자 정의 헤더 파일 "rcservo.h", "button.h", "lcd.h"

(2) 내용과 기능

(가) N_CHANNEL

ADC 변환에 사용되는 채널의 수를 나타낸다. 여기서는 4채널 사용

(나) msec_delay 함수

밀리초(millisecond) 단위로 지연(delay)을 수행하는 함수

(다) RCInit 함수

서보 모터를 초기화하는 함수로, 주어진 주기(period)에 맞게 타이머와 PWM 설정

(라) RCWidth 함수

서보 모터의 채널별 너비(width)를 설정하는 함수

(마) main 함수

주 프로그램의 진입점(entry point)이며 여기서는 버튼 입력 및 조이스틱 입력을 처리하고, 이에 따라 서보 모터의 동작을 제어하고 LCD에 정보를 표시하는 역할 수행

(3) 주된 동작

- (가) 라이브러리 및 헤더 파일을 초기화한다.
- (나) 필요한 핀을 활성화하고, 초기화 함수를 호출하여 초기 설정 수행한다.
- (다) ADC를 초기화하고, 버튼 입력 및 조이스틱 입력을 읽어 들인다.
- (라) 버튼 및 조이스틱 입력에 따라 서보 모터의 각 채널 너비 조정한다.
- (마) 각 채널의 너비가 허용 범위를 초과하지 않도록 확인하고, 서보 모터에 너비를 적용한다.
- (바) LCD에 정보 표시한다.
- (사) 상기 1-6단계 반복 실행한다.

이 코드는 AVR 마이크로컨트롤러를 사용하여 서보 모터와 버튼, 조이스틱, LCD 등을 제어하는 예시 프로그램으로 이해할 수 있다.

2. 코드 작성

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include "rcservo.h"
#include "button.h"
#include "lcd.h"

#define N_CHANNEL 4
//ADC변환 4채널을 사용한다. X축, Y축변환

void msec_delay(int n);
void RCInit(unsigned short period);
void RCWidth(unsigned short *width);

int main()
{
    unsigned short width[5]; //
    각각 차례대로 베이스, 숄더, 엘보, 손목, 그리퍼 부분이다.
    short joystick[N_CHANNEL];
    //조이스틱 X축, Y축부분이다.
    unsigned char btn;
```

```

        //버튼 변수 선언
unsigned char pressed;
//버튼을 계속 누르고 있는지 감지
char string[20];
//lcd에 표시될 값
int i;
DDRF = 0xF3;
        //간섭이 발생하므로 사용하는 핀만 활성화함

LcdInit();
        //LCD 초기화
BtnInit();
        //버튼 초기화
RCInit(10000);
        //서보모터 초기화
sei();
        //전역변수 선언

ADCSRA = (1<<ADEN) | (3<<ADPS0); // /
분주비를 8로 설정하고 ADC를 사용

width[0] = NEUTRAL_WIDTH2-100;
//베이스 중앙에 고정
width[3] = width[4] = NEUTRAL_WIDTH2; // /
손목, 그리퍼 부분 각각 중앙에 고정
width[1] = MAX_WIDTH2;
        //숄더 최대각도 2500 고정
width[2] = MIN_WIDTH2;
        //엘보 최소각도 500 고정

while(1)
{
    btn = BtnInput_Press(&pressed);
    switch(btn)
    {

```

```

case BTN_SW0:
//버튼 누르면 손목부분 각도증가

    width[3] += 100;
    break;
case BTN_SW1:
//버튼 누르면 손목부분 각도감소
    width[3] -= 100;
    break;
case BTN_SW2:
//버튼 누르면 그리퍼 각도증가
    width[4] += 100;
    break;
case BTN_SW3:
//버튼 누르면 그리퍼 각도감소
    width[4] -= 100;
    break;
default:
    break;
}
for(i=0; i<N_CHANNEL; i++)
    //AD변환 부분
{
    ADMUX = (1<<REFS0) | (i<<MUX0);
    //3.3V 사용 ADC채널 설정
    ADCSRA |= (1<<ADSC);
    //AD변환 시작
    while(!(ADCSRA&(1<<ADIF)));
    //변환 종료를 기다림
    ADCSRA |= (1<<ADIF);
    //ADIF플래그 지움
    joystick[i] = ADC;
    //변환 결과를 배열에 저장
}
joystick[2] = (int)((float)joystick[2]*267.0/1023.0); //X축 최대

```

값 180

```
joystick[3] = (int)((float)joystick[3]*267.0/1023.0); //Y축 최대
```

값 180

```
if(joystick[2] >= 120){
width[0] += 30;
}
else if(joystick[2] <= 20){
width[0] -= 30;
}
if(joystick[3] <= 20){
width[1] -= 30;
width[2] += 30;
}
else if(joystick[3] >= 160){
width[1] += 30;
width[2] -= 30;
}
if(joystick[3]>120 && joystick[2]>120)
{

}
```

```
if(width[0] > 2000) width[0] = 2000;
//베이스 최대값 이상으로 넘어갈때 최대값 고정
if(width[0] < 700) width[0] = 700;
//베이스 최소값 이상으로 넘어갈때 최소값 고정
if(width[1] > 2500) width[1] = 2500; //
숄더 최대값 이상으로 넘어갈때 최대값 고정
if(width[1] < 1000) width[1] = 1000; //
숄더 최소값 이상으로 넘어갈때 최소값 고정
if(width[2] > 2000) width[2] = 2000; //
엘보 최대값 이상으로 넘어갈때 최대값 고정
if(width[2] < 500) width[2] = 500;
//엘보 최소값 이상으로 넘어갈때 최소값 고정
```

```

        if(width[3] > 2700)                width[3] = 2700;                / /
        손목 최대값 이상으로 넘어갈때 최대값 고정
        if(width[3] < 700)                width[3] = 700;
        //손목 최소값 이상으로 넘어갈때 최소값 고정
        if(width[4] > 2700)                width[4] = 2700;                / /
        그리퍼 최대값 이상으로 넘어갈때 최대값 고정
        if(width[4] < 700)                width[4] = 700;
        //그리퍼 최소값 이상으로 넘어갈때 최소값 고정

        LcdMove(0,0); LcdPuts("JoyStick Game");

        sprintf(string, "X:%4d", joystick[2]);
        //LCD에 X축의 좌표 출력
        LcdMove(1,0); LcdPuts(string);

        sprintf(string, "Y:%4d", joystick[3]);
        //LCD에 Y축의 좌표 출력
        LcdMove(1,8); LcdPuts(string);

        RCWidth(width);
    }
}

void RCInit(unsigned short period)
{
    unsigned short oncount;
    DDRB |= 0xE0;                //베이스, 솔더, 엘보 부분 출력설정
    DDRE |= 0x18;                //손목, 그리퍼 부분 출력설정

    TCCR1A = 0xAA;                //비교일치가 될 때 OCnX = 0을 출력
    한다.
    TCCR1B = 0x18;                //FAST PWM을 설정해준다.

    TCCR3A = 0xA2;                //비교일치가 될 때 OCnX = 0을 출력

```

한다.

```
TCCR3B = 0x18; //FAST PWM을 설정해준다.
```

```
ICR1 = period *2; //카운터의 TOP을 설정해준다.
```

```
ICR3 = period *2; //카운터의 TOP을 설정해준다.
```

```
oncount = NEUTRAL_WIDTH*2;
```

```
OCR1A = oncount;
```

```
OCR1B = oncount;
```

```
OCR1C = oncount;
```

```
OCR3A = oncount;
```

```
OCR3B = oncount;
```

```
TCCR1B |= (2<<CS10); //분주비 8로 타이머 시작
```

```
TCCR3B |= (2<<CS10); //분주비 8로 타이머 시작
```

```
}
```

```
void RCWidth(unsigned short *onWidth)
```

```
{
```

```
OCR1A = onWidth[0]*2;
```

```
OCR1B = onWidth[1]*2;
```

```
OCR1C = onWidth[2]*2;
```

```
OCR3A = onWidth[3]*2;
```

```
OCR3B = onWidth[4]*2;
```

```
}
```

```
void msec_delay(int n)
```

```
{
```

```
for(; n>0; n--)
```

```
_delay_ms(1);
```

```
}
```

교수 방법

- 제어공학, 마이크로컨트롤러, 회로이론 등 기초적인 내용에 관해 잘 알고 있는지 확인하고 로봇 주제어 장치 하드웨어의 회로 설계의 검증에 관해 설명한다.
- 회로 구성 요소의 상호 관계 그리고 그 검증 방법 등에 관한 시각적 자료를 만들어 이해하기 쉽게 설명한다.
- 신호 간섭과 노이즈 파악 및 대책에 관한 다양한 사례를 쉽게 설명한다.
- 데이터 통신 회로 도면을 실제로 제작하고 관리하는 실습 과정 중에 문제점을 발견할 수 있는 상황을 가상으로 만들어 지도한다.
- 부품 간 클럭 오류나 노이즈 발생 원인과 대책에 관해 다양한 사례 수집을 유도한다.

학습 방법

- 수강의 준비를 위해 제어공학, 마이크로컨트롤러, 회로이론 등 회로 검증의 기초적인 내용에 관해 사전에 충분히 학습한다.
- 학습에 임하기 전에 회로 구성 요소의 구조와 상호 관계 등에 관한 기초 내용을 학습한다.
- 학습 전에 신호 간섭과 노이즈에 관해 미리 검색하고 정리한다.
- 데이터 통신 회로 도면을 실제로 제작하고 관리하는 실습을 견학하고 내용을 정리한다.
- 부품 간 클럭 오류나 노이즈 발생 원인과 해결책, 예방책에 관해 다양한 사례를 수집하고 연구한다.

평가 준거

- 평가자는 학습자가 학습 목표를 성공적으로 달성하였는지를 평가해야 한다.
- 평가자는 다음 사항을 평가해야 한다.

학습 내용	학습 목표	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 검증	- 개념설계 단계에서 결정된 기능을 검증하기 위하여 항목, 순서, 방법을 결정할 수 있다.			
	- 전원과 클럭을 인가하여 MCU가 정상적으로 작동하는지 판단할 수 있다.			
	- 개념설계 단계에서 결정된 기능을 검증하기 위하여 MCU 하드웨어 검증코드를 작성할 수 있다.			
	- 검증작업을 수행하고 문제점을 분석하여 검증 결과 보고서를 작성할 수 있다.			

평가 방법

- 서술형 시험

학습 내용	평가 항목	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 회로 설계	- 주제어 장치의 다양한 부품의 사양을 확인하고 배치할 수 있는 능력			
	- 다양한 인터페이스를 설계하고 확인할 수 있는 능력			
	- 다양한 클럭 오류나 노이즈 등에 관한 원인, 해결책, 예방책의 숙달 여부			

• 평가자 질문

학습 내용	평가 항목	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 검증	- 주제어 장치의 다양한 부품의 사양을 확인하고 검증한 경험과 지식 보유 여부			
	- 다양한 인터페이스를 설계하고 오류를 식별할 수 있는 능력			
	- 다양한 클럭 오류나 노이즈 등에 관한 원인 파악, 해결, 예방의 숙달 여부			

• 평가자 체크리스트

학습 내용	평가 항목	성취수준		
		상	중	하
로봇 주제어 장치 (MCU) 하드웨어 검증	- 주제어 장치의 다양한 부품의 사양을 확인하고 검토할 수 있는 능력			
	- 다양한 인터페이스를 제작하고 검증한 경험의 보유 여부			
	- 다양한 클럭 오류나 노이즈 등에 관한 원인 분석, 해결책, 예방책의 경험 보유 여부			

피드백

1. 서술형 시험 - 회로 검증에서 필수적인 부분과 선택적인 부분을 구분하여 서술형 문제를 제출한 후, 시험을 치르도록 한다. - 서술형 시험 결과를 평가하고 그 결과를 확인할 수 있도록 한다. - 평가 결과 성적이 저조한 학생은 주제어 장치회로 설계 검증에 관한 보고서 제출 기회를 부여한다. - 고득점 학습자에게는 최신 설계 동향에 관해 안내하고 보다 선진화된 기법을 인식할 수 있게 한다. 2. 평가자 질문 - 회로 검증에 관한 질문 목록을 만들어 학습 시간에 피평가자에게 질문을 한 후 답변에 관하여 조언한다. - 답변을 잘하지 못한 학생은 주제어 장치 회로 설계와 검증에 관한 보고서 제출 기회를 부여한다. - 고득점 학습자에게는 최신 설계 기법을 소개한다. 3. 평가자 체크리스트 - 회로를 설계하고 검증하기 위하여 수집된 자료 품질에 관한 수준을 서로 비교하고 평가한 결과를 공유한다. - 체크리스트 결과의 적정성, 과정 등을 평가하고 미비점을 지적한 후 추가 학습 결과를 제출하도록 한다. - 고득점 학습자에게는 AI에 관한 개념과 사례를 소개한다.



- 민세아(2014. 10. 8.), “클라우드에서 내려와 지상으로! 포그 컴퓨팅”, 보안뉴스.
- ASUS IoT 홈페이지. “ASUS TINKER BOARD SERIES”. <https://tinker-board.asus.com/accessories/tinker-kit.html>에서 2023. 06. 02. 검색.
- BinGoon(2013년2월5일). “블루투스 TTL과 CMOS간 레벨변환 보드만들기, HC-06 블루투스 모듈설정”. <https://binworld.kr/m/37>에서 2023. 05. 28. 검색.
- BinGoon(2013년2월9일). “블루투스 로봇 메인보드 만들기, AVR 하드웨어 컨트롤러 보드 설계”. <https://binworld.kr/m/39>에서 2023. 05. 28. 검색.
- congP. “STM32 MCU 전원회로설계 가이드 및 주의 사항”. 콩이가 알려 주는 IT Blog. <https://vuzwatistory.com/entry/STM32-MCU-%EC%A0%84%EC%9B%90%ED%9A%8C%EB%A1%9C%EC%84%A4%EA%B3%84-%EA%B0%80%EC%9D%B4%EB%93%9C-%EB%B0%8F-%EC%A3%BC%EC%9D%98%EC%82%AC%ED%95%AD#:~:text=VDD%EC%97%90%20%2B1.8~.3.3V%EB%A5%BC%20%EC%97%B0%EA%B2%B0%ED%95%98%EA%B3%A0%20VSS%EC%97%90%20GND%EB%A5%BC%20%EC%97%B0%EA%B2%B0%ED%95%98%EB%A9%B4%20%EB%90%9C%EB%8B%A4.%20%ED%8F%AC%ED%8A%B8%EC%97%90,%EC%A0%84%EC%9B%90%EA%B3%B5%EA%B8%89%EC%9D%84%20%EB%8B%B4%EB%8B%B9%ED%95%98%EB%8A%94%20%EB%82%B4%EB%B6%80%20LDO%20%28Voltage%20regulator%29%EC%97%90%20%EA%B3%B5%EA%B8%89%EB%90%98%EB%8A%94%20%EC%A0%84%EC%9B%90%EC%9D%B4%EB%8B%A4>에서 2023. 06. 15. 검색

NCS학습모듈 개발이력

발행일	2016년 12월 31일		
세분류명	로봇하드웨어설계(19030801)		
개발기관	부경대학교 산학협력단, 한국직업능력개발원		
집필진	주문갑(부경대학교)*	검토진	김종형(서울과학기술대학교)
	이경창(부경대학교)		노재상(에이오비)
	이경창(부경대학교)		박영제(성균관대학교)
	이현규(시스템베이스)		전상원(파인로보틱스)
	진태석(동서대학교)		이래찬(대광테크)
*표시는 대표집필자임			
발행일	2018년 12월 31일		
학습모듈명	로봇 MCU 하드웨어 회로 설계(LM1903080120_16v2)		
개발기관	대진대학교 산학협력단(개발책임자: 백창화), 한국직업능력연구원		
발행일	2023년 12월 31일		
학습모듈명	로봇 주제어 장치(MCU) 하드웨어 회로 설계(LM1903080120_22v3)		
개발기관	대진대학교 산학협력단(개발책임자: 백창화), 한국직업능력연구원		
집필진	인한수(크로스젠)*	검토진	김영균(엔씨링크)
*표시는 대표집필자임			

로봇 주제어 장치(MCU) 하드웨어 회로 설계(LM1903080120_22v3)

저작권자	교육부
연구기관	한국직업능력연구원
발행일	2023. 12. 31.
ISBN	979-11-6961-975-2

※ 이 학습모듈은 자격기본법 시행령(제8조 국가직무능력표준의 활용)에 의거하여 개발하였으며, NCS통합포털사이트(<http://www.ncs.go.kr>)에서 다운로드 할 수 있습니다.



www.ncs.go.kr